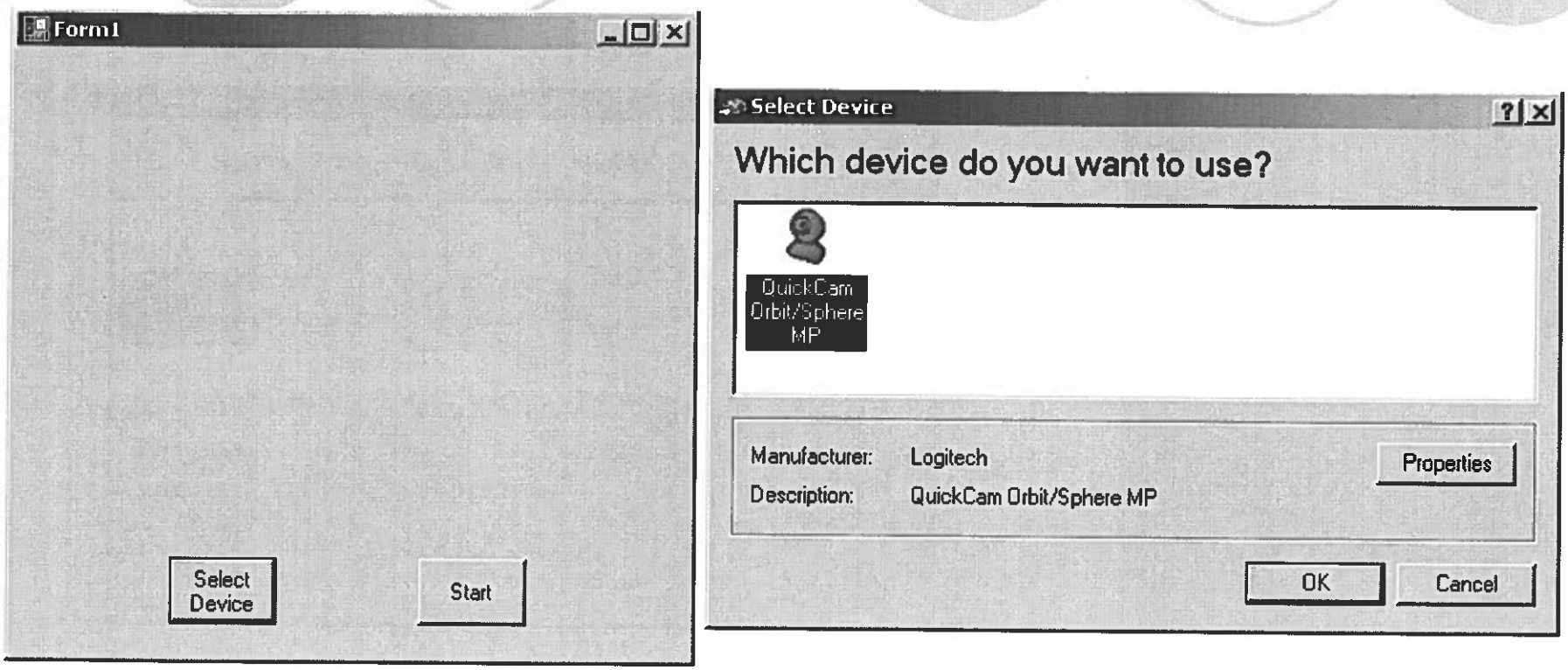
3D Video Conferencing

Senior Project

Philip Andal

Jeremy Tibayan

Capturing video feed

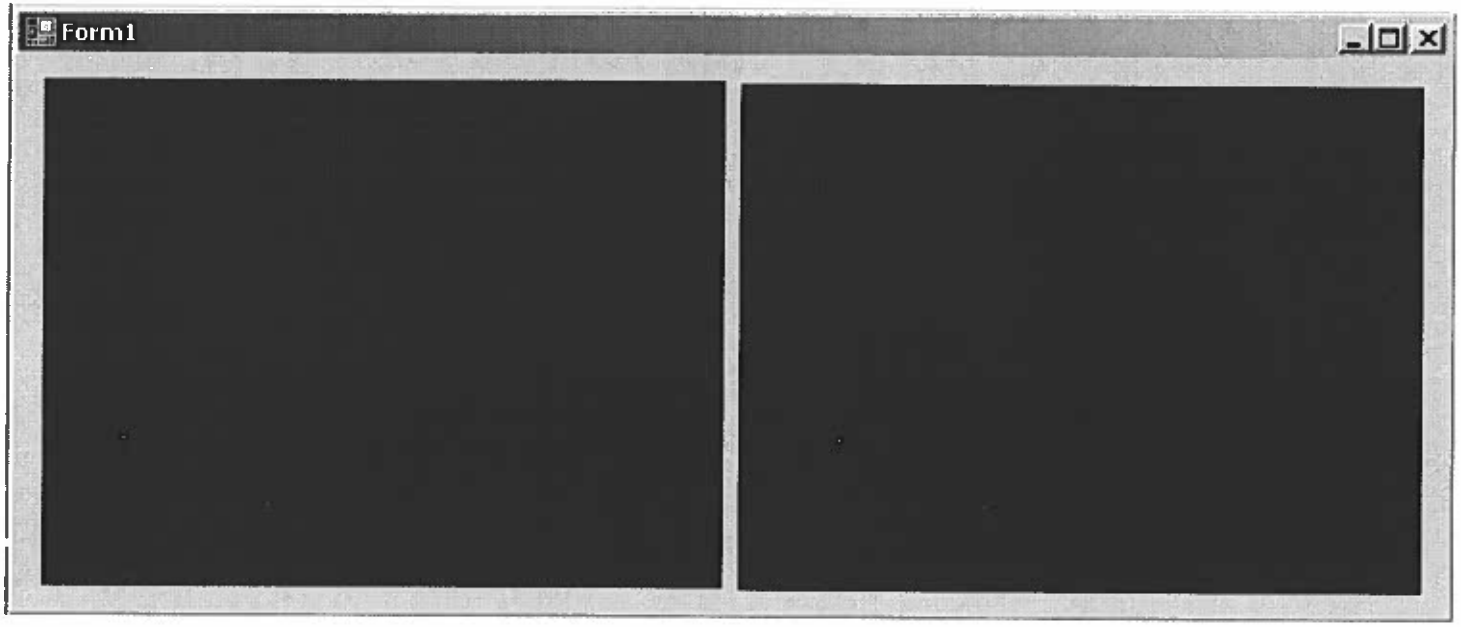


Video feed shows up in the left form above

Goals

- October 1 – get the feed to show up
- October 8 – get the feed from 2 cameras
- Start working with the 3D glasses

2 Video Feeds (Automatic)



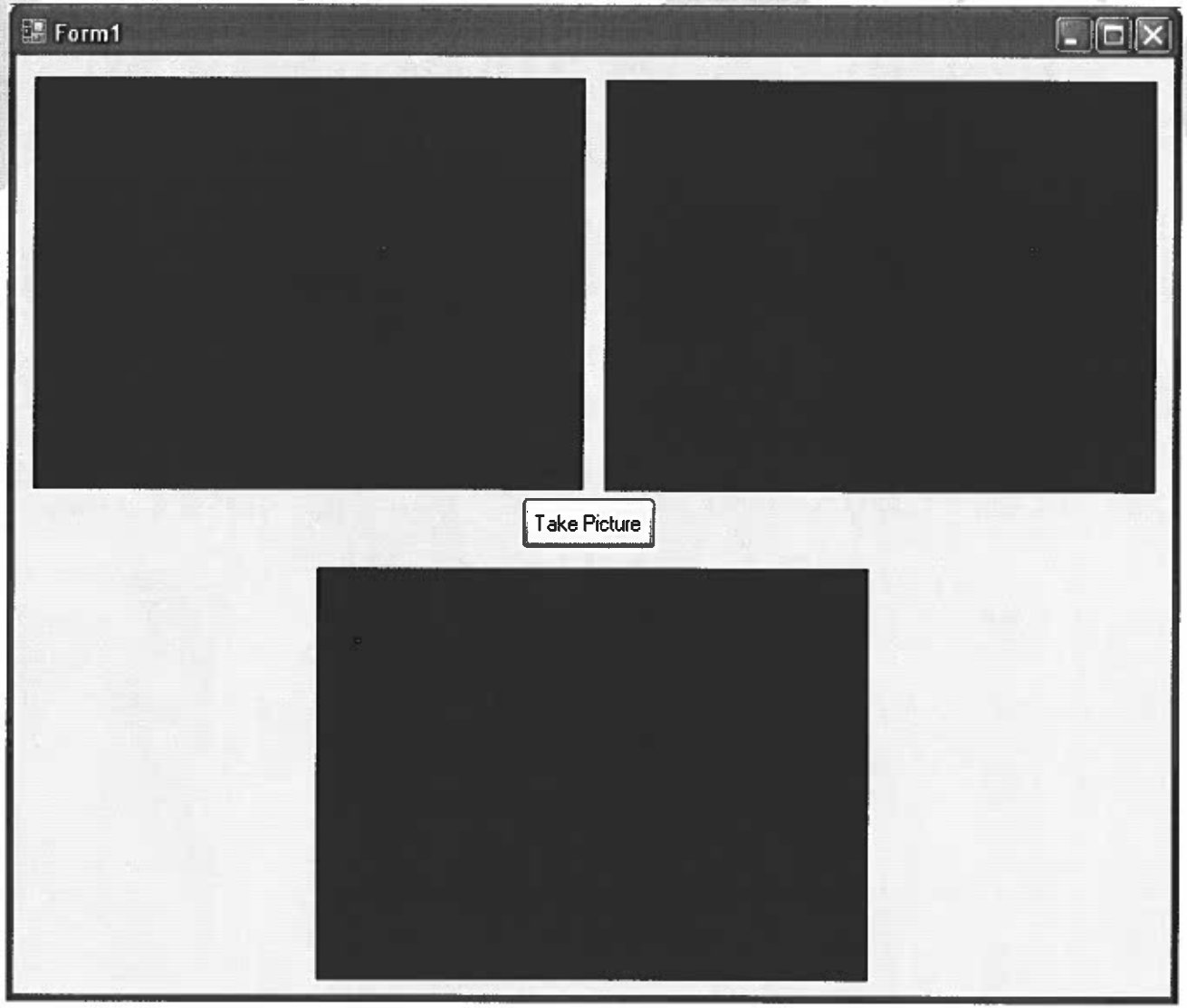
Left cam in left panel, right cam in right panel

Goals

- More research on 3D Video Glasses
- Start with the networking portion with single client
- Wait for 3D glasses

Blending Images

- To prepare ourselves for blending video feeds (more difficult), we started off with blending two images.



Bottom panel contains blended image (not video)

Goals

- Experiment with 3D Video Glasses
- Continue with the networking portion with single client
- Continue working on blending video feeds

Problems/Roadblocks

- WIA Library
  - No way to capture video
  - Capturing image by image is way too slow

Solutions/Workarounds

- Use AVICap class
  - An API, no managed class for .NET
  - Need to use P/Invoke to make use of AVICap
  - Examples using AVICap captured video at < 15 fps
- Use C++
  - May be very difficult
- Use DirectShowNet Library
  - Most likely going to be using this

Right now…

- Working on the client/server part of the project.
- Close to sending pictures

Goals

- Finish working on the client/server portion of our project.
- Continue working on capturing and encoding video feeds from the web cameras.

Right now …

- Sending image through local network using UDP.
- Demonstration.

Schedule

- Deciding on a DShow.net method.
  - Combine DxWebCam sample with send-picture program.
  - 1 week.
- Design protocol.
  - 2-3 weeks.
- Interleave and stream video frames with chosen encoding method.
  - 4-5 weeks.
- Sync with 3D glasses and put it all together.
  - Documentation.
  - 2-3 weeks.

Right now…

- Trying simple method
  - Capturing image from form every *x* milliseconds (may be too slow)
- Working with Dshow.NET methods for more fluid video

Goals

- Continue working on capturing and encoding video feeds from the web cameras.
- Continue working on network protocol.

Demonstration

Goals

- Sync two threads to interleave the two video feeds.
- Sync another thread to integrate voice.
- Optional: text messages.

3D Video Conferencing

Developers: Philip Andal & Jeremy Tibayan

Advisor: Dr. Lin

Project Introduction

- Want to create a more immersive experience for users participating in a video conference

Problem

- "Immersiveness"
  1) Eye contact - Conference members aren't really making eye contact
  2) Flat/2D image – Feeling of "being there" is not there

Our Solution

- Eye contact – no solution from us, others working on technology to fix this problem
- **Flat/2D Image – Interleaving video frames from two cameras paired with LCD shutter glasses**

Why this project?

- Make 3D video conferencing possible for personal or business use
- Also make it more accessible/affordable for anyone seeking a better video conferencing experience

Application

- Examples:
  - Normal users can use it for a more immersive experience with friends and family
  - Teachers can use a variation of it to lecture a group of students
  - Military can use it so that soldiers overseas can have a more personal means of communication

Network

- UDP is an unreliable connectionless network protocol
  - Many packets are dropped or come in the wrong order
  - Need to tolerate dropped and unordered packets to be able to use over the internet
- One of the reasons why we're currently limited to a local network

Video Codec (cont.)

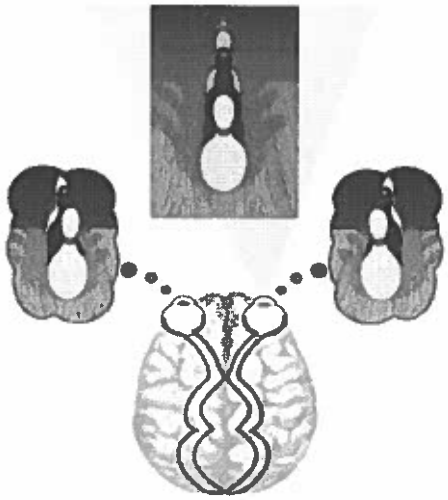
- MJPEG (Motion JPEG)
  - All frames are Intra or Key frames
    - This is not suitable for the internet. Local network is fine
- WMV (Windows Media Video)
  - Windows Media Encoder SDK for .NET framework doesn't contain anything useful for proper video conferencing. It would have to record the whole stream on the sender side

Video Codec (cont.)

- H.264 – jointly created by ITU-T Video Coding Experts Group (VCEG) and ISO/IEC Moving Picture Experts Group (MPEG)
  - ITU-T: Telecommunication Standardization Sector of the International Telecommunication Union
  - ISO/IEC : International Organization for Standardization and International Electrotechnical Commission
  - Widely used in video conferencing applications
    - Used in conjunction with RTP (Real-time Transport Protocol)
- With H.264 and WMV we couldn't find a way to have both cams linked to one stream

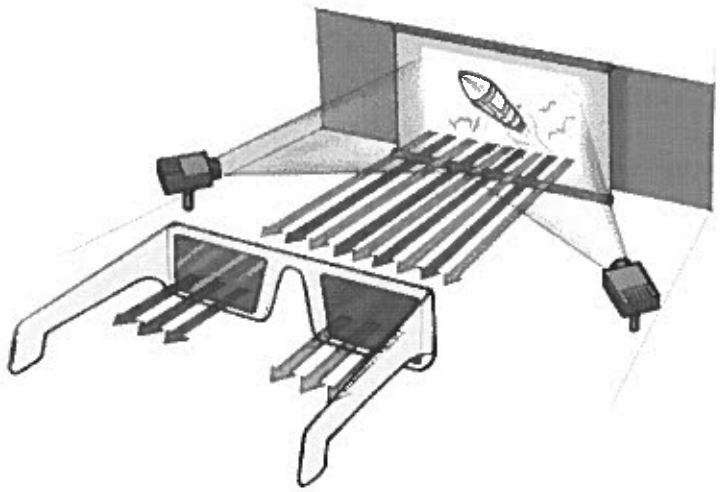
Optics

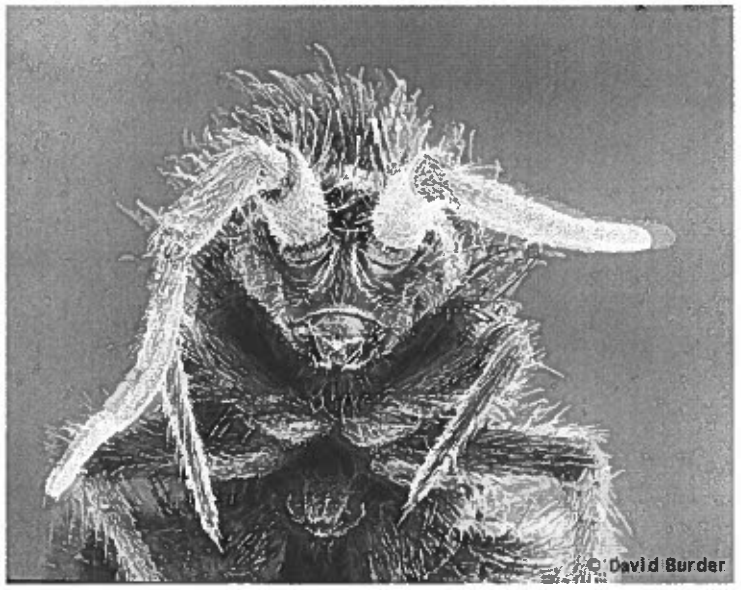
- How our vision works:
  - We have 2 eyes and each eye sees the same area but at different angles (cover one eye at a time and notice the difference)
  - The brain combines these two images and perceives the differences as depth
  - Without this depth perception everything would look 2D



Different Techniques

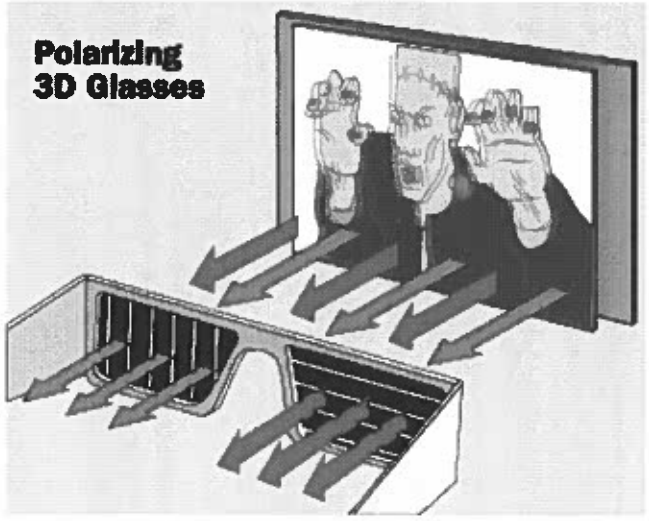
- Traditional Red/Blue filtered glasses
  - You have 2 different angles of one image. One angle in red and the other in blue.
  - The red filter of the glasses blocks red angle and receives the blue while the blue filter blocks out the blue angle and receives the red.





Different Techniques (cont.)

- Polarized glasses
  - There are 2 projections of an image at different angles. Each projection has different polarization
  - The glasses contain two filtered lenses with different polarizations. Each lens allows only the light of one polarization through and blocks the other (orthogonal) polarized light



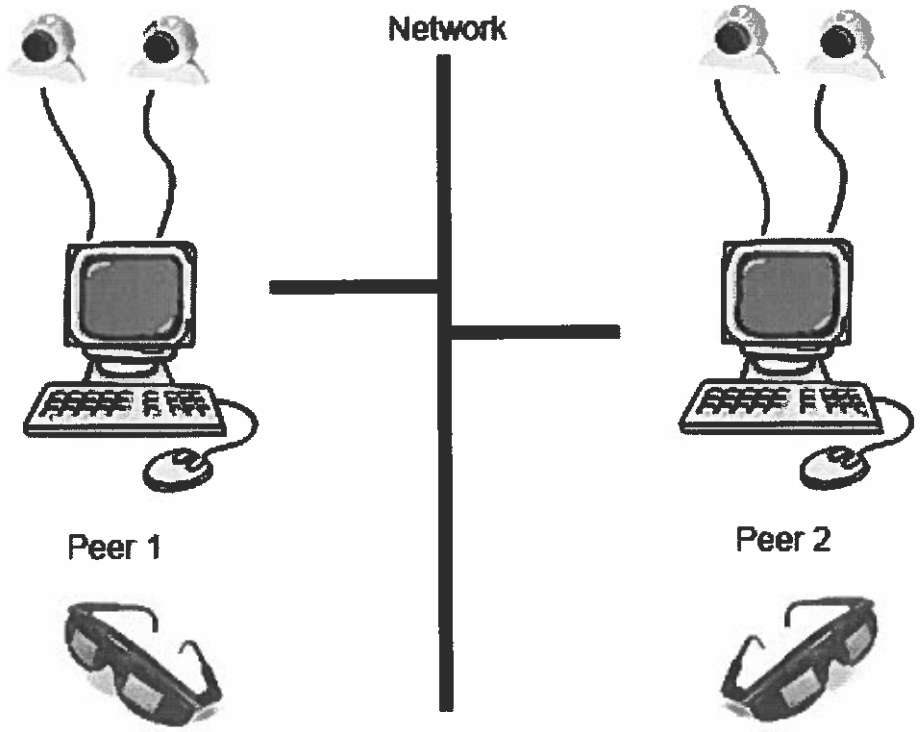
Different Techniques (cont.)

- LCD shutter glasses (the technique we used)
  - You have 2 different angles of the same image constantly interleaved
  - The glasses, which is in sync with the interleaved images, rapidly blocks one eye when receiving one angle and blocks the other eye when receiving the other angle

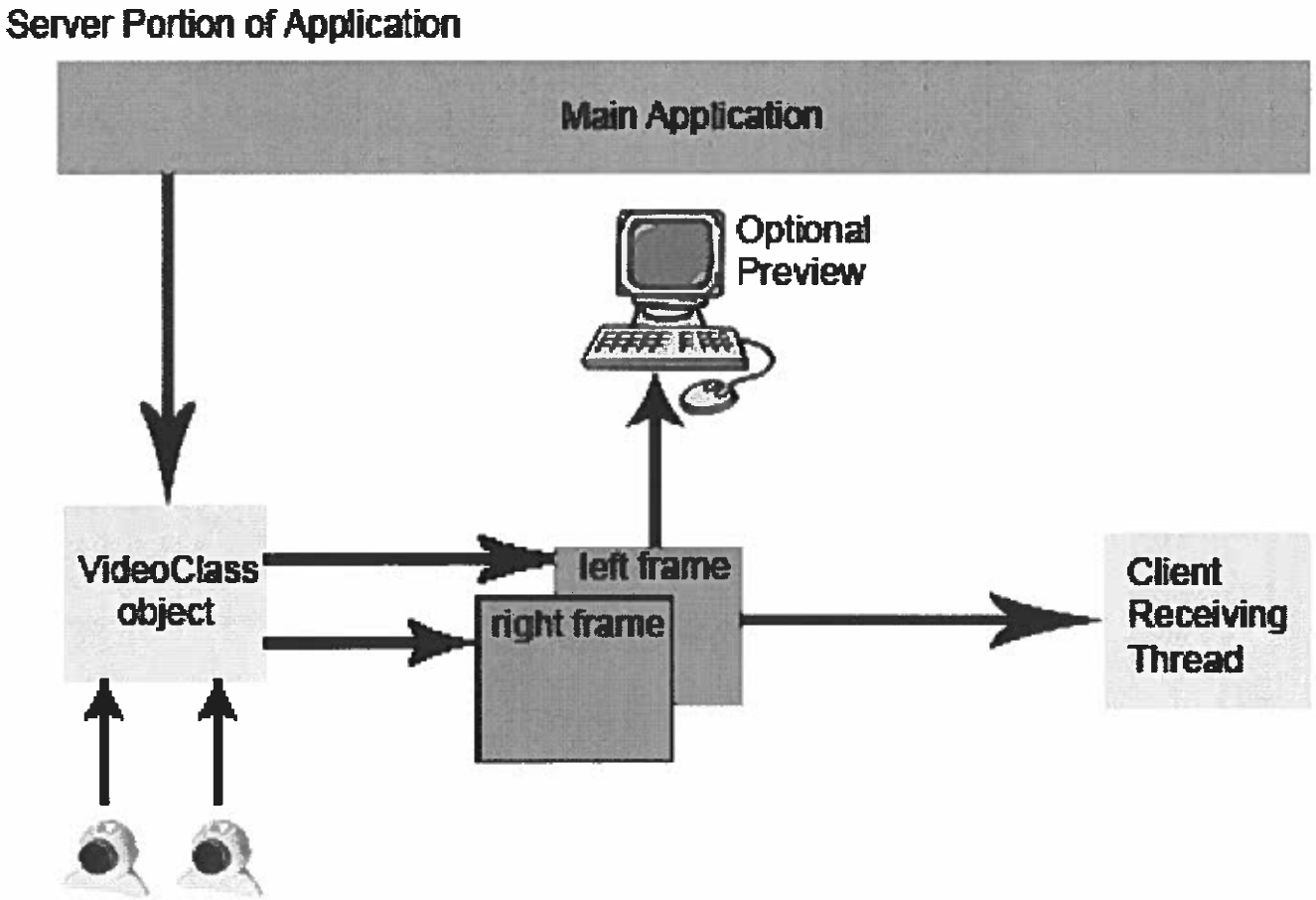
Hardware and Software

- Hardware
  - LCD Shutter Glasses
  - Two DirectShow compatible webcams (most webcams are)
- Software
  - DirectShowNet Library
    - Open source DirectShow wrapper for the .NET framework
  - BASS.Net API for voice

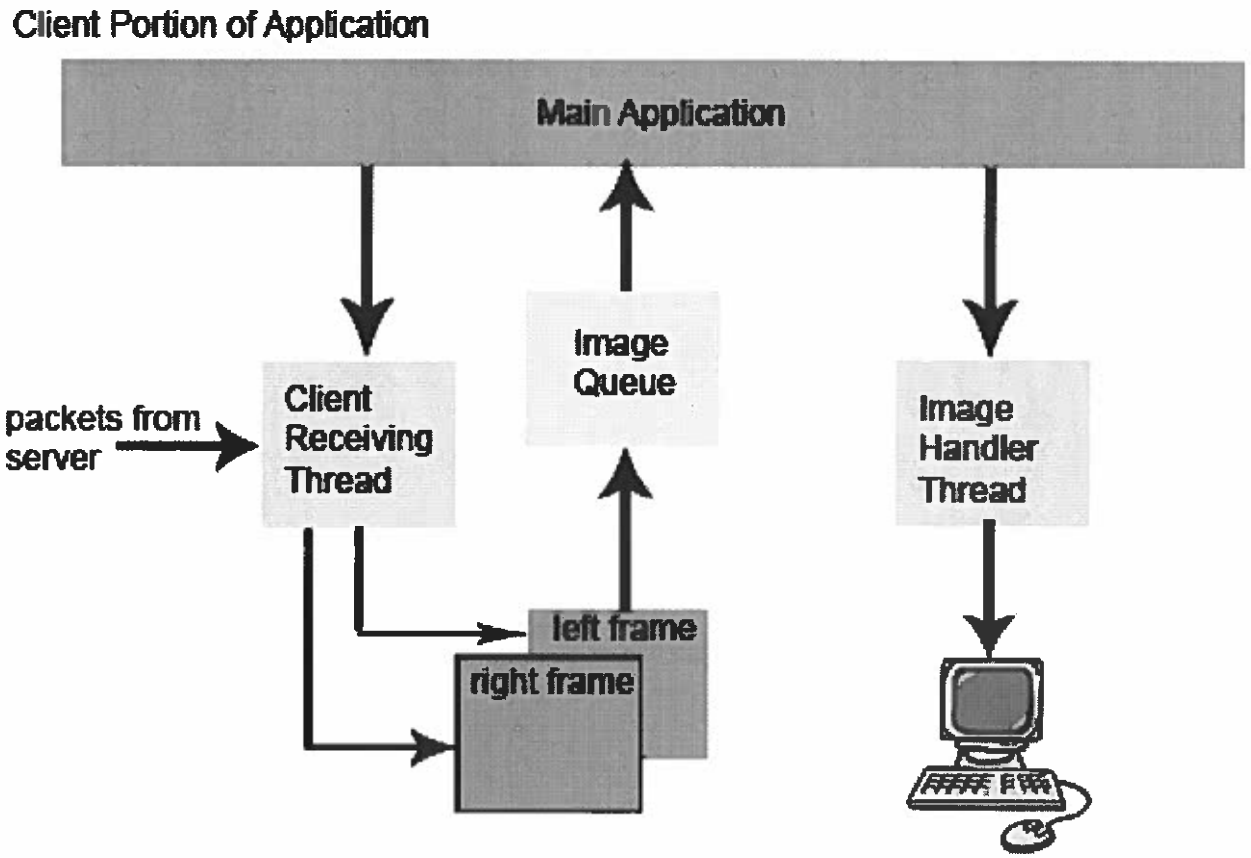
Typical Setup



Data Flow (Simplified)



Data Flow (Simplified)



Demonstration

Future Development/Ideas

- Make it usable over the internet
  - Use a better codec more suitable for the internet
  - Keep in mind dropped and unordered packets
    - Use of RTP would greatly help this area
- IR emitter (capture the IR signal from eDimensional dongle, replicate it and emit through IR emitter)
- Multicast – allow multiple clients in a video conference
- Face tracking – people usually don't stay still
- Auto Focus/Angle – when movement occurs or when camera spacing changes
- Dynamic resolution – video dimensions adapts to what the network can handle

Achievements

- Our project is merely a stepping stone
- Able to interleave frames through local network
- Established a base for further research
- Almost able to see a 3D image
  - Syncing the glasses exactly with the program will solve this

Summary

- Problem
  - Regular video conferencing uses 2D images. Wanted a more immersive experience (3D video)
- Solution
  - LCD shutter glasses with interleaving images from 2 webcams creating an illusion of 3D
- A lot can still be done

References

- http://www.3dglassesonline.com/how%2Ddo%2D3d%2Dglasses%2Dwork/
- http://www.vision3d.com/stereo.html
- http://www.wikipedia.org
- http://www.codeguru.com (forums)
- http://www.codeproject.com/ (many good examples)
- http://www.un4seen.com/
- http://forums.microsoft.com/msdn/
- http://msdn.microsoft.com/
- http://www.newmediarepublic.com/dvideo/compression/adv07.html
- http://directshownet.sourceforge.net/
- http://www.dur.ac.uk/n.s.holliman/Presentations/DaveWoodhouse.pdf (research paper on 3D video games)
- http://www.informikon.com/directshow-tutorials

```
/*********************************************************************

Form1.cs

This is the main form. Everything is done within this form

Controls:

   Buttons:
    btn_send - this button starts sending video from the capture
    devices

    btnSlide - this button slides the right side of the form out
    to reveal video conferencing and other options

    button1 - this button stops all outgoing audio/video packets

   Check Boxes:
    check3D - this checkbox is to disable/enable 3D video

    previewCheck - this checkbox is to disable/enable a small
    preview window during video chat

    voiceCheck - this checkbox is to disable/enable voice

   Tool Strip Menu Items:
    exitToolStripMenuItem - exits the program and frees all
    resources

    fileToolStripMenuItem - Opens the file menu

    settingsToolStripMenuItem - opens the settings form

   Text Boxes:
    ipBox - text box for remote IP user wants to chat with

    richTextBox1 - text box for outgoing and incoming text
    messages

    textMsgBox - text box for text messages to be sent by
    user input

   Picture Boxes:
    pictureBox3 - main picture box. Displays incoming video

    leftPreview - shows preview of outgoing video from left cam

    rightPreview - shows preview of outgoing video from right cam

*********************************************************************/

using System;
using System.Collections;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Drawing.Imaging;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.Net;
using System.Net.Sockets;
using System.IO;
using System.Threading;
using System.Runtime.InteropServices;
```

```
using System.Diagnostics;

namespace _3D_Video
{
    public partial class Form1 : Form
    {

        #region member variables

        //capture devices and voice objects
        private VideoClass vid;
        private VoiceClass voice;

        //constant variables
        private const int FRAMERATE = 30, HANGUP = -1, CALL = 1, INTERVAL = 33, FREQ =
    44100;
        private const string LOCALHOST = "127.0.0.1";

        //queue received images
        private Queue<Image> img_q = new Queue<Image>();

        //memory stream for received images
        private static MemoryStream in_ms;

        //UDP Socket to send all outgoing data (text, image, voice)
        private Socket udpSocket = new Socket(AddressFamily.InterNetwork, SocketType.
    Dgram, ProtocolType.Udp);

        //Data (text, image, voice) receiving threads
        private Thread msgRecvThread;
        private Thread recvVoiceThread;
        private Thread recvThread; //image receiving thread

        //IP string variable
        private static string ipAddress;

        //Timers
        private System.Windows.Forms.Timer myTimer = new System.Windows.Forms.Timer();//
    send images timer
        private System.Timers.Timer dispImgTimer = new System.Timers.Timer(33.333333);//
    display images timer
        private System.Windows.Forms.Timer slideOutTimer = new System.Windows.Forms.Timer
    ();//formslide timers
        private System.Windows.Forms.Timer slideInTimer = new System.Windows.Forms.Timer
    ();

        //create form variable to for Settings
        private System.Windows.Forms.Form settingsForm;

        //font properties for text messages
        private Font stdFont, idFont;

        //booleans (mainly for checkboxes)
        private static bool exitFlag, camsOn, voiceON, preview;

        //image quality variable
        private long IMG_QUALITY;

        //image variable for preview boxes
        private static Image image;

        //control for the main form (Form1)
        private static Control mainForm;

        //count for form slider
        private static int sizecount = 0;
```

```
        //variables set to the preview picture boxes and the received image picture box
        private static PictureBox[] preBox = new PictureBox[2];
        private static PictureBox[] pBox = new PictureBox[1];

        #endregion member variables

        //delegate functions for cross threaded calls
        delegate void dispMsgCallback(string msg, bool id, Color c);
        delegate void dispRemIPCallback(string ip);
        delegate void dispPicCallback(MemoryStream ms);



        public Form1()
        {
            InitializeComponent();

            //enable double buffer to reduce flicker
            this.SetStyle(ControlStyles.UserPaint | ControlStyles.AllPaintingInWmPaint | ↙
    ControlStyles.OptimizedDoubleBuffer, true);

            //set mainForm to control form properties
            mainForm = TopLevelControl;
        }

        #region OnLoad

        //initializations on program load
        private void Form1_Load(object sender, EventArgs e)
        {
            //ipAddress defaulted to 127.0.0.1
            ipAddress = LOCALHOST;

            //init video class
            vid = new VideoClass(Properties.Settings.Default.LeftCam, Properties.Settings↙
    .Default.RightCam, FRAMERATE);
            vid.video3D(true);

            //init voice class
            voice = new VoiceClass(Properties.Settings.Default.audioIn, Properties.      ↙
    Settings.Default.audioOut, FREQ);
            voice.Init();

            //all booleans initialized to false
            exitFlag = camsOn = preview = voiceON = false;
            pBox[0] = pictureBox3;

            //timers initializations
            myTimer.Tick += new EventHandler(SendVideo);
            myTimer.Interval = INTERVAL;

            dispImgTimer.Elapsed += new System.Timers.ElapsedEventHandler(dispImgEvent);
            dispImgTimer.Start();


            //receiving threads initializations (create all as background and start)
            recvThread = new Thread(new ThreadStart(receivingThread));
            recvThread.IsBackground = true;
            recvThread.Start();

            msgRecvThread = new Thread(new ThreadStart(textReceiving));
            msgRecvThread.IsBackground = true;
            msgRecvThread.Start();

            recvVoiceThread = new Thread(new ThreadStart(recvVoice));
```

```
            recvVoiceThread.IsBackground = true;
            recvVoiceThread.Start();

            //button and fonts for text messaging
            idFont = new Font("Microsoft Sans Serif", (float)9, FontStyle.Bold);
            stdFont = new Font("Microsoft Sans Serif", (float)9, FontStyle.Regular);
            btnText.Enabled = false;

            //set image quality according to loaded settings
            IMG_QUALITY = Properties.Settings.Default.ImageQuality;

            //preview boxes
            preBox[0] = leftPreview;
            preBox[1] = rightPreview;

        }
        #endregion OnLoad


        //function to display remote IP in ipBox from any receiving thread
        public void dispRemIP(string ip)
        {

            if (richTextBox1.InvokeRequired)
            {
                dispRemIPCallback d = new dispRemIPCallback(dispRemIP);
                Invoke(d, new object[] { ip });
            }
            else
            {
                ipBox.Text = ip;
            }

        }

        //function that hides/displays specified buttons
        private static void hideShowButton(Button b)
        {
            b.Visible = !b.Visible;
        }


        #region Video

        //function to display received image from receiving thread (not used)
        public void dispPic(MemoryStream ms)
        {

            if (pBox[0].InvokeRequired)
            {
                dispPicCallback d = new dispPicCallback(dispPic);
                this.Invoke(d, new object[] { ms });
            }
            else
            {
                pBox[0].Image = Image.FromStream(ms);
            }

        }

        //looks at image queue and displays first image
        private void dispImgEvent(object o, System.Timers.ElapsedEventArgs e)
        {
            if (img_q.Count > 1)
            {
                pBox[0].Image = img_q.Dequeue();
```

```
            }
        }

        //thread to receive incoming images
        public void receivingThread()
        {
            //create UDP Client to receive images
            UdpClient udpClient = new UdpClient(Properties.Settings.Default.vPort);
            IPEndPoint remEP = new IPEndPoint(IPAddress.Any, 0);

            while (true)
            {
                //create new memory stream
                in_ms = new MemoryStream();

                //block receive: waits for image to be received and stores it in buffer
                byte[] buffer = udpClient.Receive(ref remEP);

                //display IP from which image is received
                if (ipBox.Text != remEP.Address.ToString())
                    dispRemIP(remEP.Address.ToString());

                //writer data in buffer to memory stream
                in_ms.Write(buffer, 0, buffer.Length);

                try
                {
                    //dispPic(in_ms);

                    //enqueue memory stream object into the image queue
                    if (img_q.Count < 30)
                        img_q.Enqueue(Image.FromStream(in_ms));
                }
                catch //reaches here if received data is not an image
                {     //therefore either remote user has hung up or and error occured
                    try
                    {
                        //convert non-image data to an integer: if -1, user has hung up
                        if (BitConverter.ToInt32(buffer, 0) == HANGUP)
                        {
                            dispMsg("\n***" + remEP.Address.ToString() + " stopped sending video feed ***\n\n", true, Color.DarkOrange);

                            //clear what's ever remaining in the image queue
                            img_q.Clear();
                        }

                        //call setup ***Not Working***
                        #region forCallSetup
                        if (BitConverter.ToInt32(buffer, 0) == CALL)
                        {
                            dispMsg("\n***" + remEP.Address.ToString() + " wants to send a video feed ***\n\n", true, Color.DarkOrange);
                            if (MessageBox.Show("Would you like to accept?", "Call", MessageBoxButtons.YesNo) == DialogResult.Yes)
                                udpClient.Send(BitConverter.GetBytes(CALL), sizeof(int), remEP);
                            else
                                udpClient.Send(BitConverter.GetBytes(HANGUP), sizeof(int), remEP);
                        }
                        #endregion
                    }
                    catch (Exception e)
                    {
                        MessageBox.Show(e.Message.ToString());
```

```
                }
            }
        }
    }

    //event when "Stop" button is pressed
    private void button1_Click(object sender, EventArgs e)
    {
        //hide the button
        hideShowButton(button1);

        //stop video and voice classes as we as set flags
        exitFlag = true;
        vid.Stop();
        camsOn = false;
        voice.Stop();

    }

    //event when send is clicked
    private void btn_send_Click(object sender, EventArgs e)
    {
        //hide button
        hideShowButton(button1);

        //if the ipBox is not empty, store string in ipAddress
        //otherwise default to localhost
        if (ipBox.Text != "")
            ipAddress = ipBox.Text;
        else
            ipAddress = LOCALHOST;

        //check is cams aren't on, turn them on
        if (!camsOn)
            StartCams();

        //if user wants voice, call voice function
        if (voiceON)
            StartVoice();

        //set exitFlag to false
        exitFlag = false;

        //start timer to send images
        myTimer.Start();

        //Socket udpSocket = new Socket(AddressFamily.InterNetwork, SocketType.Dgram, ↙
ProtocolType.Udp);
        //IPEndPoint iep = new IPEndPoint(IPAddress.Parse(ipAddress), 8080);
        //int i = udpSocket2.SendTo(BitConverter.GetBytes(CALL), iep);
        //callThread.Start(udpSocket2);


    }

    //Call-Setup: *** DOES NOT WORK***
    #region CallSetupd(unused)
    //private Thread callThread;
    //private static byte[] accepted = new byte[4];
    //private void receiveCallThread(object o)
    //{
    //    IPEndPoint iep = new IPEndPoint(IPAddress.Parse(ipAddress), 8080);
    //    ((Socket)o).SendTo(BitConverter.GetBytes(CALL), iep);

    //    ((Socket)o).Receive(accepted);
```

```
        //    if (BitConverter.ToInt32(accepted, 0) == HANGUP)
        //    {
        //        ((Socket)o).Close();

        //        exitFlag = true;
        //    }
        //    else
        //    {
        //        ((Socket)o).Close();
        //        myTimer.Start();

        //    }



        //}
        #endregion CallSetupd(unused)

        //target function for timer event handler
        private void SendVideo(Object myObject, EventArgs myEventArgs)
        {
            try
            {

                if (!exitFlag)
                {
                    #region more callSetup
                    //if (!inCall)
                    //{
                    //    myTimer.Stop();
                    //    int i = udpSocket.SendTo(BitConverter.GetBytes(CALL), iep);
                    //    callThread.Start(myTimer);

                    //}
                    //else
                    //{
                    #endregion

                    //image quality in settings
                    IMG_QUALITY = Properties.Settings.Default.ImageQuality;

                    //set image quality in video class
                    vid.setQuality(IMG_QUALITY);

                    //send next image in capture graph from correct device
                    vid.Send(ipAddress, Properties.Settings.Default.vPort);

                    //if preview box is checked, show preview
                    if (preview)
                    {
                        int i; //temporary integer for preview box selection
                        image = vid.getPreview(out i);
                        preBox[i].Image = image.GetThumbnailImage(107, 80, null, IntPtr. ↙
    Zero);
                    }

                    //}
                }
                else
                {
                    //IP end point to send hangup falg
                    IPEndPoint iep = new IPEndPoint(IPAddress.Parse(ipAddress),          ↙
    Properties.Settings.Default.vPort);
                    udpSocket.SendTo(BitConverter.GetBytes(HANGUP), iep);
                    myTimer.Stop();
                }
```

```
        }
        catch (Exception e)
        {
            //udpSocket.Close();
            myTimer.Stop();
            MessageBox.Show(e.Message.ToString());
            exitFlag = true;

        }
    }

    //start capture graphs of capture devices
    private void StartCams()
    {
        //set left and right cams in case of settings change
        vid.setLeft(Properties.Settings.Default.LeftCam);
        vid.setRight(Properties.Settings.Default.RightCam);
        vid.Start();
        camsOn = true;
    }

    #endregion Video

    //changle booleans if checkboxes are changed
    #region CheckBoxes

    private void check3D_CheckedChanged(object sender, EventArgs e)
    {
        vid.video3D(check3D.Checked);
    }

    private void voiceCheck_CheckedChanged(object sender, EventArgs e)
    {
        voiceON = voiceCheck.Checked;
    }

    private void previewCheck_CheckedChanged(object sender, EventArgs e)
    {
        preview = previewCheck.Checked;
    }

    #endregion CheckBoxes

    #region TEXT

    //if the text messagebox contains text enable send button
    private void textMsgBox_TextChanged(object sender, EventArgs e)
    {
        if (textMsgBox.Text.Length > 0)
            btnText.Enabled = true;
        else
            btnText.Enabled = false;
    }

    //when enter key is pressed signal text message send button event
    private void textMsgBox_KeyDown(object sender, KeyEventArgs e)
    {
        if (e.KeyCode == Keys.Enter && e.Modifiers == Keys.None)
        {
            btnText_Click(sender, e);
            e.SuppressKeyPress = true;
        }
    }

    //preview for when enter button is pressed
```

```
        private void textMsgBox_PreviewKeyDown(object sender, PreviewKeyDownEventArgs e)
        {
            if (e.KeyCode == Keys.Enter && e.Modifiers == Keys.None)
                e.IsInputKey = true;
        }

        //display sent message and then send message
        private void btnText_Click(object sender, EventArgs e)
        {
            //if nothing in textbox don't do anything
            if (textMsgBox.Text == "")
                return;

            //display message with font settings
            dispMsg("Me: ", true, Color.DarkBlue);
            dispMsg(textMsgBox.Text + "\n", false, Color.Black);

            //send message
            textSend(textMsgBox.Text);

            //clear input text box
            textMsgBox.Text = "";
        }

        //display received message
        public void dispMsg(string msg, bool id, Color c)
        {

            if (richTextBox1.InvokeRequired)
            {
                dispMsgCallback d = new dispMsgCallback(dispMsg);
                Invoke(d, new object[] { msg, id, c });
            }
            else
            {
                richTextBox1.SelectionStart = richTextBox1.TextLength;
                if (id)
                    richTextBox1.SelectionFont = idFont;
                else
                    richTextBox1.SelectionFont = stdFont;
                richTextBox1.SelectionColor = c;
                richTextBox1.SelectedText = msg;
                richTextBox1.SelectionStart = richTextBox1.TextLength;
                richTextBox1.ScrollToCaret();
            }

        }

        //text message receiving thread
        private void textReceiving()
        {
            //create UDP Client to receive text message
            UdpClient udpClient = new UdpClient(Properties.Settings.Default.tPort);
            IPEndPoint remEP = new IPEndPoint(IPAddress.Any, Properties.Settings.Default.↙
    tPort);

            while (true)
            {
                //store message in buffer
                byte[] buffer = udpClient.Receive(ref remEP);

                //put remote IP in ipbox
                if (ipBox.Text != remEP.Address.ToString())
                    dispRemIP(remEP.Address.ToString());

                //convert and display received message and display the IP that sent it
```

```
                string remoteIP = remEP.Address.ToString()+ ": ";
                string msgStr = System.Text.ASCIIEncoding.ASCII.GetString(buffer);
                dispMsg(remoteIP, true, Color.DarkRed);
                dispMsg(msgStr + "\n", false, Color.Black);


            }


        }

        //send text message
        public void textSend(string str)
        {
            if (ipBox.Text != "")
                ipAddress = ipBox.Text;
            else
                ipAddress = LOCALHOST;
            try
            {
                IPEndPoint iep = new IPEndPoint(IPAddress.Parse(ipAddress), Properties.
    Settings.Default.tPort);

                //convert and send text message
                udpSocket.SendTo(System.Text.ASCIIEncoding.ASCII.GetBytes(str), iep);
            }
            catch (Exception e)
            {
                MessageBox.Show(e.Message.ToString());
                ipBox.Text = "";
            }


        }

        #endregion TEXT


        #region VOICE

        //start sending voice packets
        private void StartVoice()
        {
            voice.Send(ipAddress, 8060);
        }

        //receiving voice thread
        private void recvVoice()
        {
            UdpClient udpClient = new UdpClient(Properties.Settings.Default.aPort);
            while (true)
            {
                IPEndPoint remEP = new IPEndPoint(IPAddress.Any, Properties.Settings.
    Default.aPort);

                //output received buffer to speakers
                voice.PlayBuffer(udpClient.Receive(ref remEP));
            }
        }


        #endregion VOICE

        #region WindowSlide

        //UI Feature: sliding out
```

```
        private void btnSlide_Click(object sender, EventArgs e)
        {
            beginSlide();
        }

        private void beginSlide()
        {
            btnSlide.Enabled = false;
            if (btnSlide.Text == ">")
            {
                btnSlide.Text = "<";
                slideOutTimer.Tick += new EventHandler(slideOut);
                slideOutTimer.Interval = 50;
                slideOutTimer.Start();
            }
            else
            {
                btnSlide.Text = ">";
                slideInTimer.Tick += new EventHandler(slideIn);
                slideInTimer.Interval = 50;
                slideInTimer.Start();
            }
        }

        private void slideOut(Object myObject, EventArgs myEventArgs)
        {
            if (sizecount <= 17)
            {
                mainForm.Width += 20;
                sizecount++;
            }
            else
            {
                slideOutTimer.Stop();
                btnSlide.Enabled = true;
            }

        }
        private void slideIn(Object myObject, EventArgs myEventArgs)
        {
            if (sizecount > 0)
            {
                mainForm.Width -= 20;
                sizecount--;
            }
            else
            {
                slideInTimer.Stop();
                btnSlide.Enabled = true;
            }

        }

        #endregion WindowSlide


        #region FileMenu



        private void exitToolStripMenuItem_Click(object sender, EventArgs e)
        {
            exitProc();
            Application.Exit();
        }
        private void settingsToolStripMenuItem_Click(object sender, EventArgs e)
```

```
        {
            settingsForm = new SettingsForm();
            settingsForm.Show();
        }

        #endregion FileMenu



        //exit procedure
        private void exitProc()
        {
            exitFlag = true;

            //release resources taken by capture devices
            if (camsOn)
                vid.Stop();

            //abort all threads
            recvThread.Abort();
            msgRecvThread.Abort();
            recvVoiceThread.Abort();

            //stop voice
            voice.Stop();
        }

        #region Closing

        private void Form1_Closing(object sender, System.ComponentModel.CancelEventArgs ↙
    e)
        {
            exitProc();
        }

        #endregion Closing




    }
}
```

```
namespace _3D_Video
{
    partial class Form1
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed;
    otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            this.pictureBox3 = new System.Windows.Forms.PictureBox();
            this.button1 = new System.Windows.Forms.Button();
            this.label2 = new System.Windows.Forms.Label();
            this.btn_send = new System.Windows.Forms.Button();
            this.ipBox = new System.Windows.Forms.TextBox();
            this.menuStrip1 = new System.Windows.Forms.MenuStrip();
            this.fileToolStripMenuItem = new System.Windows.Forms.ToolStripMenuItem();
            this.settingsToolStripMenuItem = new System.Windows.Forms.ToolStripMenuItem()
    ;
            this.exitToolStripMenuItem = new System.Windows.Forms.ToolStripMenuItem();
            this.check3D = new System.Windows.Forms.CheckBox();
            this.voiceCheck = new System.Windows.Forms.CheckBox();
            this.textMsgBox = new System.Windows.Forms.TextBox();
            this.btnText = new System.Windows.Forms.Button();
            this.richTextBox1 = new System.Windows.Forms.RichTextBox();
            this.btnSlide = new System.Windows.Forms.Button();
            this.leftPreview = new System.Windows.Forms.PictureBox();
            this.rightPreview = new System.Windows.Forms.PictureBox();
            this.previewCheck = new System.Windows.Forms.CheckBox();
            ((System.ComponentModel.ISupportInitialize)(this.pictureBox3)).BeginInit();
            this.menuStrip1.SuspendLayout();
            ((System.ComponentModel.ISupportInitialize)(this.leftPreview)).BeginInit();
            ((System.ComponentModel.ISupportInitialize)(this.rightPreview)).BeginInit();
            this.SuspendLayout();
            //
            // pictureBox3
            //
            this.pictureBox3.Location = new System.Drawing.Point(397, 29);
            this.pictureBox3.Name = "pictureBox3";
            this.pictureBox3.Size = new System.Drawing.Size(320, 240);
            this.pictureBox3.TabIndex = 2;
            this.pictureBox3.TabStop = false;
            //
            // button1
            //
```

```
            this.button1.BackgroundImage = global::_3D_Video.Properties.Resources.
    stopbutton;
            this.button1.Image = global::_3D_Video.Properties.Resources.stopbutton;
            this.button1.Location = new System.Drawing.Point(647, 335);
            this.button1.Name = "button1";
            this.button1.Size = new System.Drawing.Size(63, 20);
            this.button1.TabIndex = 19;
            this.button1.UseVisualStyleBackColor = true;
            this.button1.Visible = false;
            this.button1.Click += new System.EventHandler(this.button1_Click);
            //
            // label2
            //
            this.label2.AutoSize = true;
            this.label2.Image = global::_3D_Video.Properties.Resources.ip_1;
            this.label2.Location = new System.Drawing.Point(12, 32);
            this.label2.Name = "label2";
            this.label2.Size = new System.Drawing.Size(19, 13);
            this.label2.TabIndex = 17;
            this.label2.Text = "    ";
            //
            // btn_send
            //
            this.btn_send.BackgroundImage = global::_3D_Video.Properties.Resources.
    sendbutton2;
            this.btn_send.Image = global::_3D_Video.Properties.Resources.sendbutton2;
            this.btn_send.Location = new System.Drawing.Point(648, 335);
            this.btn_send.Name = "btn_send";
            this.btn_send.Size = new System.Drawing.Size(63, 20);
            this.btn_send.TabIndex = 15;
            this.btn_send.UseVisualStyleBackColor = true;
            this.btn_send.Click += new System.EventHandler(this.btn_send_Click);
            //
            // ipBox
            //
            this.ipBox.Location = new System.Drawing.Point(36, 29);
            this.ipBox.Name = "ipBox";
            this.ipBox.Size = new System.Drawing.Size(330, 20);
            this.ipBox.TabIndex = 14;
            //
            // menuStrip1
            //
            this.menuStrip1.Items.AddRange(new System.Windows.Forms.ToolStripItem[] {
            this.fileToolStripMenuItem});
            this.menuStrip1.Location = new System.Drawing.Point(0, 0);
            this.menuStrip1.Name = "menuStrip1";
            this.menuStrip1.Size = new System.Drawing.Size(391, 24);
            this.menuStrip1.TabIndex = 22;
            this.menuStrip1.Text = "menuStrip1";
            //
            // fileToolStripMenuItem
            //
            this.fileToolStripMenuItem.DropDownItems.AddRange(new System.Windows.Forms.
    ToolStripItem[] {
            this.settingsToolStripMenuItem,
            this.exitToolStripMenuItem});
            this.fileToolStripMenuItem.Name = "fileToolStripMenuItem";
            this.fileToolStripMenuItem.Size = new System.Drawing.Size(35, 20);
            this.fileToolStripMenuItem.Text = "File";
            //
            // settingsToolStripMenuItem
            //
            this.settingsToolStripMenuItem.Name = "settingsToolStripMenuItem";
            this.settingsToolStripMenuItem.Size = new System.Drawing.Size(124, 22);
            this.settingsToolStripMenuItem.Text = "Settings";
            this.settingsToolStripMenuItem.Click += new System.EventHandler(this.
```

```
    settingsToolStripMenuItem_Click);
            //
            // exitToolStripMenuItem
            //
            this.exitToolStripMenuItem.Name = "exitToolStripMenuItem";
            this.exitToolStripMenuItem.Size = new System.Drawing.Size(124, 22);
            this.exitToolStripMenuItem.Text = "Exit";
            this.exitToolStripMenuItem.Click += new System.EventHandler(this.
    exitToolStripMenuItem_Click);
            //
            // check3D
            //
            this.check3D.AutoSize = true;
            this.check3D.BackgroundImage = global::_3D_Video.Properties.Resources.
    _3dvideo_1;
            this.check3D.Checked = true;
            this.check3D.CheckState = System.Windows.Forms.CheckState.Checked;
            this.check3D.Location = new System.Drawing.Point(647, 278);
            this.check3D.Name = "check3D";
            this.check3D.Size = new System.Drawing.Size(77, 17);
            this.check3D.TabIndex = 23;
            this.check3D.Text = "                    ";
            this.check3D.UseVisualStyleBackColor = true;
            this.check3D.CheckedChanged += new System.EventHandler(this.
    check3D_CheckedChanged);
            //
            // voiceCheck
            //
            this.voiceCheck.AutoSize = true;
            this.voiceCheck.BackgroundImage = global::_3D_Video.Properties.Resources.
    voice_f;
            this.voiceCheck.BackgroundImageLayout = System.Windows.Forms.ImageLayout.
    Center;
            this.voiceCheck.Location = new System.Drawing.Point(647, 311);
            this.voiceCheck.Name = "voiceCheck";
            this.voiceCheck.Size = new System.Drawing.Size(71, 17);
            this.voiceCheck.TabIndex = 24;
            this.voiceCheck.Text = "                  ";
            this.voiceCheck.UseVisualStyleBackColor = true;
            this.voiceCheck.CheckedChanged += new System.EventHandler(this.
    voiceCheck_CheckedChanged);
            //
            // textMsgBox
            //
            this.textMsgBox.Location = new System.Drawing.Point(12, 311);
            this.textMsgBox.Multiline = true;
            this.textMsgBox.Name = "textMsgBox";
            this.textMsgBox.ScrollBars = System.Windows.Forms.ScrollBars.Vertical;
            this.textMsgBox.Size = new System.Drawing.Size(312, 46);
            this.textMsgBox.TabIndex = 25;
            this.textMsgBox.TextChanged += new System.EventHandler(this.
    textMsgBox_TextChanged);
            this.textMsgBox.PreviewKeyDown += new System.Windows.Forms.
    PreviewKeyDownEventHandler(this.textMsgBox_PreviewKeyDown);
            this.textMsgBox.KeyDown += new System.Windows.Forms.KeyEventHandler(this.
    textMsgBox_KeyDown);
            //
            // btnText
            //
            this.btnText.BackgroundImage = global::_3D_Video.Properties.Resources.
    sendbutton1;
            this.btnText.Location = new System.Drawing.Point(324, 311);
            this.btnText.Name = "btnText";
            this.btnText.Size = new System.Drawing.Size(42, 44);
            this.btnText.TabIndex = 26;
            this.btnText.UseVisualStyleBackColor = true;
```

```
            this.btnText.Click += new System.EventHandler(this.btnText_Click);
            //
            // richTextBox1
            //
            this.richTextBox1.BackColor = System.Drawing.SystemColors.Window;
            this.richTextBox1.Location = new System.Drawing.Point(12, 55);
            this.richTextBox1.Name = "richTextBox1";
            this.richTextBox1.ReadOnly = true;
            this.richTextBox1.ScrollBars = System.Windows.Forms.RichTextBoxScrollBars.
    Vertical;
            this.richTextBox1.Size = new System.Drawing.Size(353, 256);
            this.richTextBox1.TabIndex = 27;
            this.richTextBox1.Text = "";
            //
            // btnSlide
            //
            this.btnSlide.BackgroundImage = global::_3D_Video.Properties.Resources.
    webcambutton2;
            this.btnSlide.BackgroundImageLayout = System.Windows.Forms.ImageLayout.Center
    ;
            this.btnSlide.Image = global::_3D_Video.Properties.Resources.webcambutton2;
            this.btnSlide.Location = new System.Drawing.Point(365, 170);
            this.btnSlide.Name = "btnSlide";
            this.btnSlide.Size = new System.Drawing.Size(27, 26);
            this.btnSlide.TabIndex = 28;
            this.btnSlide.Text = ">";
            this.btnSlide.UseVisualStyleBackColor = true;
            this.btnSlide.Click += new System.EventHandler(this.btnSlide_Click);
            //
            // leftPreview
            //
            this.leftPreview.Location = new System.Drawing.Point(397, 275);
            this.leftPreview.Name = "leftPreview";
            this.leftPreview.Size = new System.Drawing.Size(107, 80);
            this.leftPreview.TabIndex = 29;
            this.leftPreview.TabStop = false;
            //
            // rightPreview
            //
            this.rightPreview.Location = new System.Drawing.Point(510, 275);
            this.rightPreview.Name = "rightPreview";
            this.rightPreview.Size = new System.Drawing.Size(107, 80);
            this.rightPreview.TabIndex = 30;
            this.rightPreview.TabStop = false;
            //
            // previewCheck
            //
            this.previewCheck.AutoSize = true;
            this.previewCheck.BackgroundImage = global::_3D_Video.Properties.Resources.
    preview;
            this.previewCheck.Location = new System.Drawing.Point(647, 294);
            this.previewCheck.Name = "previewCheck";
            this.previewCheck.Size = new System.Drawing.Size(71, 17);
            this.previewCheck.TabIndex = 31;
            this.previewCheck.Text = "                ";
            this.previewCheck.UseVisualStyleBackColor = true;
            this.previewCheck.CheckedChanged += new System.EventHandler(this.
    previewCheck_CheckedChanged);
            //
            // Form1
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.BackgroundImage = global::_3D_Video.Properties.Resources.background_32;
            this.ClientSize = new System.Drawing.Size(391, 367);
            this.Controls.Add(this.previewCheck);
```

```
            this.Controls.Add(this.rightPreview);
            this.Controls.Add(this.leftPreview);
            this.Controls.Add(this.btnSlide);
            this.Controls.Add(this.richTextBox1);
            this.Controls.Add(this.btnText);
            this.Controls.Add(this.textMsgBox);
            this.Controls.Add(this.voiceCheck);
            this.Controls.Add(this.check3D);
            this.Controls.Add(this.button1);
            this.Controls.Add(this.label2);
            this.Controls.Add(this.btn_send);
            this.Controls.Add(this.ipBox);
            this.Controls.Add(this.pictureBox3);
            this.Controls.Add(this.menuStrip1);
            this.FormBorderStyle = System.Windows.Forms.FormBorderStyle.Fixed3D;
            this.MainMenuStrip = this.menuStrip1;
            this.Name = "Form1";
            this.Text = "Video Conference";
            this.Load += new System.EventHandler(this.Form1_Load);
            ((System.ComponentModel.ISupportInitialize)(this.pictureBox3)).EndInit();
            this.menuStrip1.ResumeLayout(false);
            this.menuStrip1.PerformLayout();
            ((System.ComponentModel.ISupportInitialize)(this.leftPreview)).EndInit();
            ((System.ComponentModel.ISupportInitialize)(this.rightPreview)).EndInit();
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.PictureBox pictureBox3;
        private System.Windows.Forms.Button button1;
        private System.Windows.Forms.Label label2;
        private System.Windows.Forms.Button btn_send;
        private System.Windows.Forms.TextBox ipBox;
        private System.Windows.Forms.MenuStrip menuStrip1;
        private System.Windows.Forms.ToolStripMenuItem fileToolStripMenuItem;
        private System.Windows.Forms.ToolStripMenuItem exitToolStripMenuItem;
        private System.Windows.Forms.CheckBox check3D;
        private System.Windows.Forms.CheckBox voiceCheck;
        private System.Windows.Forms.ToolStripMenuItem settingsToolStripMenuItem;
        private System.Windows.Forms.TextBox textMsgBox;
        private System.Windows.Forms.Button btnText;
        private System.Windows.Forms.RichTextBox richTextBox1;
        private System.Windows.Forms.Button btnSlide;
        private System.Windows.Forms.PictureBox leftPreview;
        private System.Windows.Forms.PictureBox rightPreview;
        private System.Windows.Forms.CheckBox previewCheck;
    }
}
```

```
/*************************************************************************

SettingsForm.cs

This form is for all the settings the application uses.

Settings:

   Tabs:
    Ports - this tab has the port values for text messaging, video
    conferencing, and the audio port. All ports must be different
    from each other.

    Cameras - this tab contains drop boxes for the left and right
    capture devices. There is also a quality setting to determine
    the quality of the video

    Audio - this tab contains drop boxes for audio input/output
    devices.


*************************************************************************/

using System;
using System.Collections;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using DirectShowLib;
using Un4seen.Bass.AddOn.Enc;
using Un4seen.Bass;
using Un4seen.Bass.Misc;


namespace _3D_Video
{
    public partial class SettingsForm : Form
    {
        #region Member Variables

        private bool changed = false;
        private ArrayList deviceList;
        private const string portError = "Ports must be between 1024 and 65535";

        private struct DeviceStruct
        {
            public int devNum;
            public string devName;

            //initialize
            public DeviceStruct(int d, string n)
            {
                devNum = d;
                devName = n;
            }

            //override ToString for DeviceStruct
            public override string ToString()
            {
                return devName;
            }
        }
```

```
        #endregion Member Variables

        public SettingsForm()
        {
            InitializeComponent();
            LoadDropBoxes();
            ApplySettings();
        }

        //Load the camera and audio setting drop boxes with devices
        private void LoadDropBoxes()
        {
            deviceList = getDevices();
            BASS_DEVICEINFO[] audoDevs = Bass.BASS_GetDeviceInfos();
            BASS_DEVICEINFO[] audiDevs = Bass.BASS_RecordGetDeviceInfos();

            //add all capture devices to each drop box
            for (int i = 0; i < deviceList.Count; i++)
            {
                lcDrop.Items.Add(((DeviceStruct)deviceList[i]));
                rcDrop.Items.Add(((DeviceStruct)deviceList[i]));
            }

            //add all audio devices to each drop box
            foreach (BASS_DEVICEINFO aud in audoDevs)
            {
                aoDropBox.Items.Add(aud.name);
            }

            foreach (BASS_DEVICEINFO aud in audiDevs)
            {
                aiDropBox.Items.Add(aud.name);
            }
        }

        //function to get devices
        public ArrayList getDevices()
        {
            DsDevice[] capDevices;

            capDevices = DsDevice.GetDevicesOfCat(FilterCategory.VideoInputDevice);
            ArrayList devList = new ArrayList();

            for (int i = 0; i < capDevices.Length; i++)
            {
                DeviceStruct tmp = new DeviceStruct(i, capDevices[i].Name.ToString());
                devList.Add(tmp);
            }

            return devList;
        }

        //Load last applied settings
        private void ApplySettings()
        {
            textp_tBox.Text = Properties.Settings.Default.tPort.ToString();
            vp_tBox.Text = Properties.Settings.Default.vPort.ToString();
            ap_tBox.Text = Properties.Settings.Default.aPort.ToString();
            qualityBox.Text = Properties.Settings.Default.ImageQuality.ToString();
            lcDrop.SelectedIndex = Properties.Settings.Default.LeftCam;
            rcDrop.SelectedIndex = Properties.Settings.Default.RightCam;
            aoDropBox.SelectedIndex = Properties.Settings.Default.audioOut;
            aiDropBox.SelectedIndex = Properties.Settings.Default.audioIn;
        }
```

```
        //save settings if applied
        private void SaveSettings()
        {
            try
            {
                //make sure ports are within valid range
                int i = Properties.Settings.Default.tPort = Int32.Parse(textp_tBox.Text);
                if (i < 1024 || i > 65535)
                {
                    MessageBox.Show(portError, "Error");
                    return;
                }

                i = Properties.Settings.Default.vPort = Int32.Parse(vp_tBox.Text);
                if (i < 1024 || i > 65535)
                {
                    MessageBox.Show(portError, "Error");
                    return;
                }

                i = Properties.Settings.Default.aPort = Int32.Parse(ap_tBox.Text);
                if (i < 1024 || i > 65535)
                {
                    MessageBox.Show(portError, "Error");
                    return;
                }

                if (vp_tBox.Text == textp_tBox.Text || vp_tBox.Text == ap_tBox.Text ||
ap_tBox.Text == textp_tBox.Text)
                {
                    MessageBox.Show("All ports (text, video, audio) must be unique",
"Error");
                    return;
                }

                long j = Properties.Settings.Default.ImageQuality = long.Parse(qualityBox
.Text);
                if (j < 0 || j > 100)
                {
                    MessageBox.Show("Image quality must be within 0-100 range", "Error");
                    return;
                }

                //make sure user selects unique devices
                if (((DeviceStruct)lcDrop.SelectedItem).devNum == ((DeviceStruct)rcDrop.
SelectedItem).devNum)
                {
                    MessageBox.Show("Cannot select same device for left and right cam",
"Error");
                    return;
                }
                Properties.Settings.Default.LeftCam = ((DeviceStruct)lcDrop.SelectedItem)
.devNum;
                Properties.Settings.Default.RightCam = ((DeviceStruct)rcDrop.
SelectedItem).devNum;
                Properties.Settings.Default.audioIn = aiDropBox.SelectedIndex;
                Properties.Settings.Default.audioOut = aoDropBox.SelectedIndex;
                Properties.Settings.Default.Save();
                this.Close();

            }
            catch (Exception e)
            {
                MessageBox.Show("ERROR: " + e.Message);
            }
```

```
        }
        private void applyBtn_Click(object sender, EventArgs e)
        {
            SaveSettings();
        }

        //if settings change enable "Apply" button
        private void SettingsChanged(object sender, EventArgs e)
        {
            if (!changed)
                changed = true;
            if (!applyBtn.Enabled)
                applyBtn.Enabled = true;
        }

        //close without saving changes if user press "Cancel" button
        private void cancelBtn_Click(object sender, EventArgs e)
        {
            this.Close();
        }

        private void SettingsForm_Load(object sender, EventArgs e)
        {
            applyBtn.Enabled = false;
        }



    }
}
```

```
namespace _3D_Video
{
    partial class SettingsForm
    {
        /// <summary>
        /// Required designer variable.
        /// </summary>
        private System.ComponentModel.IContainer components = null;

        /// <summary>
        /// Clean up any resources being used.
        /// </summary>
        /// <param name="disposing">true if managed resources should be disposed;
    otherwise, false.</param>
        protected override void Dispose(bool disposing)
        {
            if (disposing && (components != null))
            {
                components.Dispose();
            }
            base.Dispose(disposing);
        }

        #region Windows Form Designer generated code

        /// <summary>
        /// Required method for Designer support - do not modify
        /// the contents of this method with the code editor.
        /// </summary>
        private void InitializeComponent()
        {
            this.tabControl1 = new System.Windows.Forms.TabControl();
            this.tabPage1 = new System.Windows.Forms.TabPage();
            this.vp_tBox = new System.Windows.Forms.TextBox();
            this.textp_tBox = new System.Windows.Forms.TextBox();
            this.label6 = new System.Windows.Forms.Label();
            this.label1 = new System.Windows.Forms.Label();
            this.tabPage2 = new System.Windows.Forms.TabPage();
            this.qualityBox = new System.Windows.Forms.TextBox();
            this.label10 = new System.Windows.Forms.Label();
            this.rcDrop = new System.Windows.Forms.ComboBox();
            this.lcDrop = new System.Windows.Forms.ComboBox();
            this.label7 = new System.Windows.Forms.Label();
            this.label8 = new System.Windows.Forms.Label();
            this.applyBtn = new System.Windows.Forms.Button();
            this.cancelBtn = new System.Windows.Forms.Button();
            this.label9 = new System.Windows.Forms.Label();
            this.ap_tBox = new System.Windows.Forms.TextBox();
            this.label2 = new System.Windows.Forms.Label();
            this.tabPage3 = new System.Windows.Forms.TabPage();
            this.aiDropBox = new System.Windows.Forms.ComboBox();
            this.aoDropBox = new System.Windows.Forms.ComboBox();
            this.label3 = new System.Windows.Forms.Label();
            this.label4 = new System.Windows.Forms.Label();
            this.tabControl1.SuspendLayout();
            this.tabPage1.SuspendLayout();
            this.tabPage2.SuspendLayout();
            this.tabPage3.SuspendLayout();
            this.SuspendLayout();
            //
            // tabControl1
            //
            this.tabControl1.Controls.Add(this.tabPage1);
            this.tabControl1.Controls.Add(this.tabPage2);
            this.tabControl1.Controls.Add(this.tabPage3);
            this.tabControl1.Location = new System.Drawing.Point(12, 4);
```

```
            this.tabControl1.Name = "tabControl1";
            this.tabControl1.SelectedIndex = 0;
            this.tabControl1.Size = new System.Drawing.Size(341, 241);
            this.tabControl1.TabIndex = 0;
            //
            // tabPage1
            //
            this.tabPage1.Controls.Add(this.ap_tBox);
            this.tabPage1.Controls.Add(this.label2);
            this.tabPage1.Controls.Add(this.vp_tBox);
            this.tabPage1.Controls.Add(this.textp_tBox);
            this.tabPage1.Controls.Add(this.label6);
            this.tabPage1.Controls.Add(this.label1);
            this.tabPage1.Location = new System.Drawing.Point(4, 22);
            this.tabPage1.Name = "tabPage1";
            this.tabPage1.Padding = new System.Windows.Forms.Padding(3);
            this.tabPage1.Size = new System.Drawing.Size(333, 215);
            this.tabPage1.TabIndex = 0;
            this.tabPage1.Text = "Ports";
            this.tabPage1.UseVisualStyleBackColor = true;
            //
            // vp_tBox
            //
            this.vp_tBox.Location = new System.Drawing.Point(157, 92);
            this.vp_tBox.Name = "vp_tBox";
            this.vp_tBox.Size = new System.Drawing.Size(100, 20);
            this.vp_tBox.TabIndex = 8;
            this.vp_tBox.TextChanged += new System.EventHandler(this.SettingsChanged);
            //
            // textp_tBox
            //
            this.textp_tBox.Location = new System.Drawing.Point(157, 66);
            this.textp_tBox.Name = "textp_tBox";
            this.textp_tBox.Size = new System.Drawing.Size(100, 20);
            this.textp_tBox.TabIndex = 6;
            this.textp_tBox.TextChanged += new System.EventHandler(this.SettingsChanged);
            //
            // label6
            //
            this.label6.AutoSize = true;
            this.label6.Location = new System.Drawing.Point(92, 95);
            this.label6.Name = "label6";
            this.label6.Size = new System.Drawing.Size(59, 13);
            this.label6.TabIndex = 3;
            this.label6.Text = "Video Port:";
            //
            // label1
            //
            this.label1.AutoSize = true;
            this.label1.Location = new System.Drawing.Point(52, 69);
            this.label1.Name = "label1";
            this.label1.Size = new System.Drawing.Size(99, 13);
            this.label1.TabIndex = 0;
            this.label1.Text = "Text Message Port:";
            //
            // tabPage2
            //
            this.tabPage2.Controls.Add(this.qualityBox);
            this.tabPage2.Controls.Add(this.label10);
            this.tabPage2.Controls.Add(this.rcDrop);
            this.tabPage2.Controls.Add(this.lcDrop);
            this.tabPage2.Controls.Add(this.label7);
            this.tabPage2.Controls.Add(this.label8);
            this.tabPage2.Location = new System.Drawing.Point(4, 22);
            this.tabPage2.Name = "tabPage2";
            this.tabPage2.Padding = new System.Windows.Forms.Padding(3);
```

```
            this.tabPage2.Size = new System.Drawing.Size(333, 215);
            this.tabPage2.TabIndex = 1;
            this.tabPage2.Text = "Cameras";
            this.tabPage2.UseVisualStyleBackColor = true;
            //
            // qualityBox
            //
            this.qualityBox.Location = new System.Drawing.Point(97, 146);
            this.qualityBox.MaxLength = 3;
            this.qualityBox.Name = "qualityBox";
            this.qualityBox.Size = new System.Drawing.Size(33, 20);
            this.qualityBox.TabIndex = 8;
            this.qualityBox.TextChanged += new System.EventHandler(this.SettingsChanged);
            //
            // label10
            //
            this.label10.AutoSize = true;
            this.label10.Location = new System.Drawing.Point(24, 146);
            this.label10.Name = "label10";
            this.label10.Size = new System.Drawing.Size(42, 13);
            this.label10.TabIndex = 7;
            this.label10.Text = "Quality:";
            //
            // rcDrop
            //
            this.rcDrop.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.rcDrop.FormattingEnabled = true;
            this.rcDrop.Location = new System.Drawing.Point(97, 107);
            this.rcDrop.Name = "rcDrop";
            this.rcDrop.Size = new System.Drawing.Size(214, 21);
            this.rcDrop.TabIndex = 6;
            this.rcDrop.SelectedIndexChanged += new System.EventHandler(this.
    SettingsChanged);
            //
            // lcDrop
            //
            this.lcDrop.DropDownStyle = System.Windows.Forms.ComboBoxStyle.DropDownList;
            this.lcDrop.FormattingEnabled = true;
            this.lcDrop.Location = new System.Drawing.Point(97, 68);
            this.lcDrop.Name = "lcDrop";
            this.lcDrop.Size = new System.Drawing.Size(214, 21);
            this.lcDrop.TabIndex = 5;
            this.lcDrop.SelectedIndexChanged += new System.EventHandler(this.
    SettingsChanged);
            //
            // label7
            //
            this.label7.AutoSize = true;
            this.label7.Location = new System.Drawing.Point(24, 110);
            this.label7.Name = "label7";
            this.label7.Size = new System.Drawing.Size(74, 13);
            this.label7.TabIndex = 4;
            this.label7.Text = "Right Camera:";
            //
            // label8
            //
            this.label8.AutoSize = true;
            this.label8.Location = new System.Drawing.Point(24, 71);
            this.label8.Name = "label8";
            this.label8.Size = new System.Drawing.Size(67, 13);
            this.label8.TabIndex = 3;
            this.label8.Text = "Left Camera:";
            //
            // applyBtn
            //
            this.applyBtn.Location = new System.Drawing.Point(294, 269);
```

```
            this.applyBtn.Name = "applyBtn";
            this.applyBtn.Size = new System.Drawing.Size(59, 21);
            this.applyBtn.TabIndex = 1;
            this.applyBtn.Text = "Apply";
            this.applyBtn.UseVisualStyleBackColor = true;
            this.applyBtn.Click += new System.EventHandler(this.applyBtn_Click);
            //
            // cancelBtn
            //
            this.cancelBtn.Location = new System.Drawing.Point(229, 269);
            this.cancelBtn.Name = "cancelBtn";
            this.cancelBtn.Size = new System.Drawing.Size(59, 21);
            this.cancelBtn.TabIndex = 2;
            this.cancelBtn.Text = "Cancel";
            this.cancelBtn.UseVisualStyleBackColor = true;
            this.cancelBtn.Click += new System.EventHandler(this.cancelBtn_Click);
            //
            // label9
            //
            this.label9.AutoSize = true;
            this.label9.Font = new System.Drawing.Font("Microsoft Sans Serif", 8.25F,
    System.Drawing.FontStyle.Regular, System.Drawing.GraphicsUnit.Point, ((byte)(0)));
            this.label9.ForeColor = System.Drawing.Color.Red;
            this.label9.Location = new System.Drawing.Point(12, 248);
            this.label9.Name = "label9";
            this.label9.Size = new System.Drawing.Size(262, 13);
            this.label9.TabIndex = 7;
            this.label9.Text = "NOTE: Must restart program for settings to take effect!";
            //
            // ap_tBox
            //
            this.ap_tBox.Location = new System.Drawing.Point(157, 118);
            this.ap_tBox.Name = "ap_tBox";
            this.ap_tBox.Size = new System.Drawing.Size(100, 20);
            this.ap_tBox.TabIndex = 10;
            this.ap_tBox.TextChanged += new System.EventHandler(this.SettingsChanged);
            //
            // label2
            //
            this.label2.AutoSize = true;
            this.label2.Location = new System.Drawing.Point(92, 121);
            this.label2.Name = "label2";
            this.label2.Size = new System.Drawing.Size(59, 13);
            this.label2.TabIndex = 9;
            this.label2.Text = "Audio Port:";
            //
            // tabPage3
            //
            this.tabPage3.Controls.Add(this.aiDropBox);
            this.tabPage3.Controls.Add(this.aoDropBox);
            this.tabPage3.Controls.Add(this.label3);
            this.tabPage3.Controls.Add(this.label4);
            this.tabPage3.Location = new System.Drawing.Point(4, 22);
            this.tabPage3.Name = "tabPage3";
            this.tabPage3.Padding = new System.Windows.Forms.Padding(3);
            this.tabPage3.Size = new System.Drawing.Size(333, 215);
            this.tabPage3.TabIndex = 2;
            this.tabPage3.Text = "Audio";
            this.tabPage3.UseVisualStyleBackColor = true;
            //
            // aiDropBox
            //
            this.aiDropBox.DropDownStyle = System.Windows.Forms.ComboBoxStyle.
    DropDownList;
            this.aiDropBox.FormattingEnabled = true;
            this.aiDropBox.Location = new System.Drawing.Point(96, 107);
```

```
            this.aiDropBox.Name = "aiDropBox";
            this.aiDropBox.Size = new System.Drawing.Size(214, 21);
            this.aiDropBox.TabIndex = 10;
            this.aiDropBox.SelectedIndexChanged += new System.EventHandler(this.
    SettingsChanged);
            //
            // aoDropBox
            //
            this.aoDropBox.DropDownStyle = System.Windows.Forms.ComboBoxStyle.
    DropDownList;
            this.aoDropBox.FormattingEnabled = true;
            this.aoDropBox.Location = new System.Drawing.Point(96, 68);
            this.aoDropBox.Name = "aoDropBox";
            this.aoDropBox.Size = new System.Drawing.Size(214, 21);
            this.aoDropBox.TabIndex = 9;
            this.aoDropBox.SelectedIndexChanged += new System.EventHandler(this.
    SettingsChanged);
            //
            // label3
            //
            this.label3.AutoSize = true;
            this.label3.Location = new System.Drawing.Point(23, 110);
            this.label3.Name = "label3";
            this.label3.Size = new System.Drawing.Size(64, 13);
            this.label3.TabIndex = 8;
            this.label3.Text = "Audio Input:";
            //
            // label4
            //
            this.label4.AutoSize = true;
            this.label4.Location = new System.Drawing.Point(23, 71);
            this.label4.Name = "label4";
            this.label4.Size = new System.Drawing.Size(72, 13);
            this.label4.TabIndex = 7;
            this.label4.Text = "Audio Output:";
            //
            // SettingsForm
            //
            this.AutoScaleDimensions = new System.Drawing.SizeF(6F, 13F);
            this.AutoScaleMode = System.Windows.Forms.AutoScaleMode.Font;
            this.ClientSize = new System.Drawing.Size(364, 299);
            this.Controls.Add(this.label9);
            this.Controls.Add(this.cancelBtn);
            this.Controls.Add(this.applyBtn);
            this.Controls.Add(this.tabControl1);
            this.Name = "SettingsForm";
            this.Text = "Settings";
            this.Load += new System.EventHandler(this.SettingsForm_Load);
            this.tabControl1.ResumeLayout(false);
            this.tabPage1.ResumeLayout(false);
            this.tabPage1.PerformLayout();
            this.tabPage2.ResumeLayout(false);
            this.tabPage2.PerformLayout();
            this.tabPage3.ResumeLayout(false);
            this.tabPage3.PerformLayout();
            this.ResumeLayout(false);
            this.PerformLayout();

        }

        #endregion

        private System.Windows.Forms.TabControl tabControl1;
        private System.Windows.Forms.TabPage tabPage1;
        private System.Windows.Forms.TabPage tabPage2;
        private System.Windows.Forms.Label label6;
```

```
        private System.Windows.Forms.Label label1;
        private System.Windows.Forms.TextBox vp_tBox;
        private System.Windows.Forms.TextBox textp_tBox;
        private System.Windows.Forms.Button applyBtn;
        private System.Windows.Forms.Button cancelBtn;
        private System.Windows.Forms.Label label7;
        private System.Windows.Forms.Label label8;
        private System.Windows.Forms.ComboBox rcDrop;
        private System.Windows.Forms.ComboBox lcDrop;
        private System.Windows.Forms.Label label9;
        private System.Windows.Forms.TextBox qualityBox;
        private System.Windows.Forms.Label label10;
        private System.Windows.Forms.TextBox ap_tBox;
        private System.Windows.Forms.Label label2;
        private System.Windows.Forms.TabPage tabPage3;
        private System.Windows.Forms.ComboBox aiDropBox;
        private System.Windows.Forms.ComboBox aoDropBox;
        private System.Windows.Forms.Label label3;
        private System.Windows.Forms.Label label4;
    }
}
```

```
/*****************************************************************************
VideoClass
This class uses the Capture class to set up 2 video capture devices.
It will send interleaved images from the 2 capture devices.

Constructors and Functions:

   VideoClass(int left, int right, int iFrameRate) constructor:
    Initializes a new instance of VideoClass.  Initialized with three
    int variables: left, right, and iFrameRate. left is the index of
    the left capture device. right is the index of the right capture
    device. iFrameRate is the framerate at which both capture graphs
    of each device will run at.

   setLeft(int device), setRight(int device) functions:
    Sets the left and right devices if user wants to change capture
    devices

   video3D(bool confSetting) function:
    Sets whether the VideoClass should be sending interleaving images
    or just the image of the left capture device

   setQuality(long quality) function:
    Sets the image quality of the outgoing images

   Init() function:
    Initializes both capture devices with specified framerate

   Start() function:
    Starts both capture devices

   Pause(int dev):
    Pauses capture graph of specified device to save resources

   Stop():
    Disposes of both capture devices when not sending video

   getPreview(out int handle):
    Returns the current frame and the index of the capture device from
    which the frame is coming from

   Send(string ip, int port):
    Sends current frame to remote user of ip through port

   getImage(int device):
    Returns a memory stream of the current image of device

   GetEncoderParams(long quality), GetEncoderInfo(string mimeType) functions:
    Functions for getting quality parameters for images

   ~VideoClass()
    Class Destructor

*****************************************************************************/

using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Net;
using System.Net.Sockets;
using System.Drawing;
using System.Drawing.Imaging;
using System.IO;

namespace _3D_Video
```

```
{
    internal class VideoClass
    {
        #region Member Variables

        const int LEFT = 0, RIGHT = 1;
        int selDev;

        //Array of 2 capture devices
        Capture[] cam = new Capture[2];

        //Capture device properties
        int leftDev, rightDev, frameRate;

        //boolean for 3D or standard video conference
        bool vid3D, helper3D;

        //Video property variables
        long vidQuality;

        //Socket, ip, and port to send video
        Socket sock;
        IPEndPoint IEP;

        //Current image (for preview)
        Image currImg;

        #endregion Member Variables

        /// <summary> Initializes a new instance of the VideoClass</summary>
        public VideoClass(int left, int right, int iFrameRate)
        {
            leftDev = left;
            rightDev = right;
            frameRate = iFrameRate;
            vid3D =  helper3D = false;
            vidQuality = 100;
            sock = new Socket(AddressFamily.InterNetwork, SocketType.Dgram, ProtocolType.↙
    Udp);
            IEP = new IPEndPoint(IPAddress.Any, 0);

        }

        /// <summary> Set/Change left device</summary>
        public void setLeft(int device)
        {
            leftDev = device;
        }
        /// <summary> Set/Change right device</summary>
        public void setRight(int device)
        {
            rightDev = device;
        }

        /// <summary> Set to 2D or 3D video conferencing</summary>
        public void video3D(bool confSetting)
        {
            vid3D = confSetting;
        }

        /// <summary> Set video quality (1 - 100)</summary>
        public void setQuality(long quality)
        {
            vidQuality = quality;
        }
```

```
        /// <summary> initialize captures devices if needed. </summary>
        public void Init()
        {
            cam[LEFT] = new Capture(leftDev, frameRate);
            cam[RIGHT] = new Capture(rightDev, frameRate);
        }

        /// <summary> Start capture devices</summary>
        public void Start()
        {
            Init();
            cam[LEFT].Start();
            cam[RIGHT].Start();
        }

        /// <summary> Pause Specific capture device</summary>
        public void Pause(int dev)
        {
            cam[dev].Pause();
        }

        /// <summary> Kill capture devices</summary>
        public void Stop()
        {
            if (cam[LEFT] != null && cam[RIGHT] != null)
            {
                cam[LEFT].Dispose();
                cam[RIGHT].Dispose();
            }
        }

        /// <summary> Return current image and selected Device</summary>
        public Image getPreview(out int handle)
        {
            //tell calling function which camera (left or right) the image is coming from
            handle = selDev;

            //return current image grabbed from filter
            return currImg;
        }

        /// <summary> Send video frames to peer</summary>
        public void Send(string ip, int port)
        {
            IEP.Address = IPAddress.Parse(ip);
            IEP.Port = port;
            helper3D = !helper3D;

            //if 3D video enabled, interleave frames, otherwise show images from left cam↙
    only
            if (vid3D)
            {
                //interleaving selected device
                if (helper3D)
                    selDev = LEFT;
                else
                    selDev = RIGHT;

                //if coming from 2D setting, make sure the right cam runs
                if (!cam[RIGHT].isRunning())
                    cam[RIGHT].Start();
            }
            else
            {
                //if 3D setting is off, just grab images from left camera
                selDev = LEFT;
```

```
            //pause the right capture device to save resources
            cam[RIGHT].Pause();
        }

        sock.SendTo(getImage(selDev).ToArray(), IEP);
    }

    //Returns a stream of the captured image
    private MemoryStream getImage(int device)
    {
        MemoryStream tmp = new MemoryStream();

        //get image from selected device
        currImg = cam[device].GetBitMap();

        //save image to memory stream
        currImg.Save(tmp, GetEncoderInfo("image/jpeg"), GetEncoderParams(vidQuality));
;

        //return memory stream
        return tmp;
    }

    //Destructor
    ~VideoClass()
    {
        Stop();
        sock.Close();
    }


    #region ImageQuality

    //returns encoder parameters
    private EncoderParameters GetEncoderParams(long quality)
    {
        if (quality < 0 || quality > 100)
            throw new ArgumentOutOfRangeException("quality must be between 0 and 100.
");


        // Encoder parameter for image quality
        EncoderParameter qualityParam =
            new EncoderParameter(System.Drawing.Imaging.Encoder.Quality, quality);

        EncoderParameters encoderParams = new EncoderParameters(1);
        encoderParams.Param[0] = qualityParam;

        return encoderParams;

    }

    // Returns the image codec with the given mime type
    private ImageCodecInfo GetEncoderInfo(string mimeType)
    {
        // Get image codecs for all image formats
        ImageCodecInfo[] codecs = ImageCodecInfo.GetImageEncoders();

        // Find the correct image codec
        for (int i = 0; i < codecs.Length; i++)
            if (codecs[i].MimeType == mimeType)
                return codecs[i];
        return null;
    }
```

```
        #endregion ImageQuality

    }
}
```

```
/************************************************************************
While the underlying libraries are covered by LGPL, this sample is released
as public domain.  It is distributed in the hope that it will be useful, but
WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY
or FITNESS FOR A PARTICULAR PURPOSE.
************************************************************************/

using System;
using System.Drawing;
using System.Drawing.Imaging;
using System.Collections;
using System.Runtime.InteropServices;
using System.Threading;
using System.Diagnostics;

using DirectShowLib;


namespace _3D_Video
{
    /// <summary> Summary description for MainForm. </summary>
    internal class Capture : ISampleGrabberCB, IDisposable
    {
        #region Member variables

        /// <summary> graph builder interface. </summary>
        private IFilterGraph2 m_FilterGraph = null;
        private IMediaControl m_mediaCtrl = null;

        /// <summary> so we can wait for the async job to finish </summary>
        private ManualResetEvent m_PictureReady = null;

        /// <summary> Set by async routine when it captures an image </summary>
        private volatile bool m_bGotOne = false;

        /// <summary> Indicates the status of the graph </summary>
        private bool m_bRunning = false;

        /// <summary> Dimensions of the image, calculated once in constructor. </summary>
        private IntPtr m_handle = IntPtr.Zero;
        private int m_videoWidth;
        private int m_videoHeight;
        private int m_stride;
        public int m_Dropped = 0;

        #endregion

        #region API

        [DllImport("Kernel32.dll", EntryPoint="RtlMoveMemory")]
        private static extern void CopyMemory(IntPtr Destination, IntPtr Source, int
    Length);

        #endregion

        /// <summary> Use user-sepicified capture device and frame rate</summary>
        public Capture(int iDeviceNum, int iFrameRate)
        {
            _Capture(iDeviceNum, iFrameRate, 0, 0);
        }

        ///// <summary> release everything. </summary>
        public void Dispose()
        {
            CloseInterfaces();
            if (m_PictureReady != null)
```

```
            {
                m_PictureReady.Close();
                m_PictureReady = null;
            }
        }
        // Destructor
        ~Capture()
        {
            Dispose();
        }

        public int Width
        {
            get
            {
                return m_videoWidth;
            }
        }
        public int Height
        {
            get
            {
                return m_videoHeight;
            }
        }
        public int Stride
        {
            get
            {
                return m_stride;
            }
        }
        /// <summary> capture the next image </summary>
        //public IntPtr GetBitMap()
        public Image GetBitMap()
        {
            m_handle = Marshal.AllocCoTaskMem(m_stride * m_videoHeight);

            try
            {
                // get ready to wait for new image
                m_PictureReady.Reset();
                m_bGotOne = false;

                // If the graph hasn't been started, start it.
                //Start();

                // Start waiting
                if ( ! m_PictureReady.WaitOne(5000, false) )
                {
                    throw new Exception("Timeout waiting to get picture");
                }
            }
            catch
            {
                Marshal.FreeCoTaskMem(m_handle);
                throw;
            }

            // Got one
            Image img = new Bitmap(m_videoWidth, m_videoHeight, m_stride, PixelFormat.
    Format24bppRgb, m_handle);
            img.RotateFlip(RotateFlipType.RotateNoneFlipY);

            //free memory
            Marshal.FreeCoTaskMem(m_handle);
```

```
            return img;
        }

        // Start the capture graph
        public void Start()
        {
            if (!m_bRunning)
            {
                //int hr = m_mediaCtrl.Run();
                DsError.ThrowExceptionForHR(m_mediaCtrl.Run());
                //DsError.ThrowExceptionForHR(hr);

                m_bRunning = true;
            }
        }

        // Pause the capture graph.
        // Running the graph takes up a lot of resources.  Pause it when it
        // isn't needed.
        public void Pause()
        {
            if (m_bRunning)
            {
                int hr = m_mediaCtrl.Pause();
                DsError.ThrowExceptionForHR( hr );
                m_bRunning = false;
            }
        }


        // Internal capture
        private void _Capture(int iDeviceNum, int iFrameRate, int iWidth, int iHeight)
        {
            DsDevice[] capDevices;

            // Get the collection of video devices
            capDevices = DsDevice.GetDevicesOfCat( FilterCategory.VideoInputDevice );

            if (iDeviceNum + 1 > capDevices.Length)
            {
                throw new Exception("No video capture devices found at that index!");
            }

            try
            {
                // Set up the capture graph
                SetupGraph( capDevices[iDeviceNum], iFrameRate, iWidth, iHeight);

                // tell the callback to ignore new images
                m_PictureReady = new ManualResetEvent(false);
                m_bGotOne = true;
                m_bRunning = false;
            }
            catch
            {
                Dispose();
                throw;
            }
        }

        /// <summary> build the capture graph for grabber. </summary>
        private void SetupGraph(DsDevice dev, int iFrameRate, int iWidth, int iHeight)
        {
            int hr;

            ISampleGrabber sampGrabber = null;
```

```
            IBaseFilter capFilter = null;
            ICaptureGraphBuilder2 capGraph = null;

            // Get the graphbuilder object
            m_FilterGraph = (IFilterGraph2) new FilterGraph();
            m_mediaCtrl = m_FilterGraph as IMediaControl;


            try
            {
                // Get the ICaptureGraphBuilder2
                capGraph = (ICaptureGraphBuilder2) new CaptureGraphBuilder2();

                // Get the SampleGrabber interface
                sampGrabber = (ISampleGrabber) new SampleGrabber();

                // Start building the graph
                hr = capGraph.SetFiltergraph( m_FilterGraph );
                DsError.ThrowExceptionForHR( hr );

                // Add the video device
                hr = m_FilterGraph.AddSourceFilterForMoniker(dev.Mon, null, "Video input"↙
    , out capFilter);
                DsError.ThrowExceptionForHR( hr );

                IBaseFilter baseGrabFlt = (IBaseFilter) sampGrabber;
                ConfigureSampleGrabber(sampGrabber);

                // Add the frame grabber to the graph
                hr = m_FilterGraph.AddFilter( baseGrabFlt, "Ds.NET Grabber" );
                DsError.ThrowExceptionForHR( hr );

                // If any of the default config items are set
                if (iFrameRate + iHeight + iWidth > 0)
                {
                    SetConfigParms(capGraph, capFilter, iFrameRate, iWidth, iHeight);
                }

                hr = capGraph.RenderStream( PinCategory.Capture, MediaType.Video,        ↙
    capFilter, null, baseGrabFlt );
                DsError.ThrowExceptionForHR( hr );

                SaveSizeInfo(sampGrabber);
            }
            finally
            {
                if (capFilter != null)
                {
                    Marshal.ReleaseComObject(capFilter);
                    capFilter = null;
                }
                if (sampGrabber != null)
                {
                    Marshal.ReleaseComObject(sampGrabber);
                    sampGrabber = null;
                }
                if (capGraph != null)
                {
                    Marshal.ReleaseComObject(capGraph);
                    capGraph = null;
                }
            }
        }

        private void SaveSizeInfo(ISampleGrabber sampGrabber)
        {
```

```
            int hr;

            // Get the media type from the SampleGrabber
            AMMediaType media = new AMMediaType();
            hr = sampGrabber.GetConnectedMediaType( media );
            DsError.ThrowExceptionForHR( hr );

            if( (media.formatType != FormatType.VideoInfo) || (media.formatPtr == IntPtr.Zero) )
            {
                throw new NotSupportedException( "Unknown Grabber Media Format" );
            }

            // Grab the size info
            VideoInfoHeader videoInfoHeader = (VideoInfoHeader) Marshal.PtrToStructure( media.formatPtr, typeof(VideoInfoHeader) );
            m_videoWidth = videoInfoHeader.BmiHeader.Width;
            m_videoHeight = videoInfoHeader.BmiHeader.Height;
            m_stride = m_videoWidth * (videoInfoHeader.BmiHeader.BitCount / 8);

            DsUtils.FreeAMMediaType(media);
            media = null;
        }
        private void ConfigureSampleGrabber(ISampleGrabber sampGrabber)
        {
            AMMediaType media;
            int hr;

            // Set the media type to Video/RBG24
            media = new AMMediaType();
            media.majorType = MediaType.Video;
            media.subType   = MediaSubType.RGB24;
            media.formatType = FormatType.VideoInfo;
            hr = sampGrabber.SetMediaType( media );
            DsError.ThrowExceptionForHR( hr );

            DsUtils.FreeAMMediaType(media);
            media = null;

            // Configure the samplegrabber
            hr = sampGrabber.SetCallback( this, 1 );
            DsError.ThrowExceptionForHR( hr );
        }

        // Set the Framerate, and video size
        private void SetConfigParms(ICaptureGraphBuilder2 capGraph, IBaseFilter capFilter, int iFrameRate, int iWidth, int iHeight)
        {
            int hr;
            object o;
            AMMediaType media;

            // Find the stream config interface
            hr = capGraph.FindInterface(
                PinCategory.Capture, MediaType.Video, capFilter, typeof(IAMStreamConfig).GUID, out o );

            IAMStreamConfig videoStreamConfig = o as IAMStreamConfig;
            if (videoStreamConfig == null)
            {
                throw new Exception("Failed to get IAMStreamConfig");
            }

            // Get the existing format block
            hr = videoStreamConfig.GetFormat( out media);
            DsError.ThrowExceptionForHR( hr );
```

```
            // copy out the videoinfoheader
            VideoInfoHeader v = new VideoInfoHeader();
            Marshal.PtrToStructure( media.formatPtr, v );

            // if overriding the framerate, set the frame rate
            if (iFrameRate > 0)
            {
                v.AvgTimePerFrame = 10000000 / iFrameRate;
            }

            // if overriding the width, set the width
            if (iWidth > 0)
            {
                v.BmiHeader.Width = iWidth;
            }

            // if overriding the Height, set the Height
            if (iHeight > 0)
            {
                v.BmiHeader.Height = iHeight;
            }

            // Copy the media structure back
            Marshal.StructureToPtr( v, media.formatPtr, false );

            // Set the new format
            hr = videoStreamConfig.SetFormat( media );
            DsError.ThrowExceptionForHR( hr );

            DsUtils.FreeAMMediaType(media);
            media = null;

        }

        /// <summary> Shut down capture </summary>
        private void CloseInterfaces()
        {
            int hr;

            try
            {
                if( m_mediaCtrl != null )
                {
                    // Stop the graph
                    hr = m_mediaCtrl.Stop();
                    m_bRunning = false;
                }
            }
            catch (Exception ex)
            {
                Debug.WriteLine(ex);
            }

            if (m_FilterGraph != null)
            {
                Marshal.ReleaseComObject(m_FilterGraph);
                m_FilterGraph = null;
            }
        }

        /// <summary> sample callback, NOT USED. </summary>
        int ISampleGrabberCB.SampleCB(double SampleTime, IMediaSample pSample)
        {
            if (!m_bGotOne)
            {
```

```
                // Set bGotOne to prevent further calls until we
                // request a new bitmap.
                m_bGotOne = true;
                IntPtr pBuffer;

                pSample.GetPointer(out pBuffer);
                int iBufferLen = pSample.GetSize();

                if (pSample.GetSize() > m_stride * m_videoHeight)
                {
                    throw new Exception("Buffer is wrong size");
                }

                CopyMemory(m_handle, pBuffer, m_stride * m_videoHeight);

                // Picture is ready.
                m_PictureReady.Set();
            }

            Marshal.ReleaseComObject(pSample);
            return 0;
        }

        /// <summary> buffer callback, COULD BE FROM FOREIGN THREAD. </summary>
        int ISampleGrabberCB.BufferCB( double SampleTime, IntPtr pBuffer, int BufferLen )
        {
            if (!m_bGotOne)
            {
                // The buffer should be long enought
                if(BufferLen <= m_stride * m_videoHeight)
                {
                    // Copy the frame to the buffer
                    CopyMemory(m_handle, pBuffer, m_stride * m_videoHeight);
                }
                else
                {
                    throw new Exception("Buffer is wrong size");
                }

                // Set bGotOne to prevent further calls until we
                // request a new bitmap.
                m_bGotOne = true;

                // Picture is ready.
                m_PictureReady.Set();
            }
            else
            {
                m_Dropped++;
            }
            return 0;
        }



        public bool isRunning()
        {
            return m_bRunning;
        }


    }
}
```

```
/*************************************************************************
 This class uses the Un4seen Bass 2.4 API to record/play audio streams from/to
input/output audio devices.

Functions and Constructors:

   VoiceClass Constructor(int rDev, int oDev, int freq)
    Creates VoiceClass object. rDev is the index of the audio input
    device. oDev is the index of the audio output device. freq is
    audio frequency.

   Init()
        Initializes audio devices.

   Record()
    This function creates both the recording and playback stream. It
    also creates a new RECORDPROC which calls the recorder() function
    every time it records a sample from the audio input device

   recorder(int handle, IntPtr buffer, int length, IntPtr user)
    Reallocates the local buffer every time there is a sample callback.
    Copies data pointed to by IntPtr buffer into the local buffer and
    sends the data if the sending flag is set to true. Returns true
    if no errors occured

   PlayBuffer(byte[] buffer)
    Takes data in the buffer and outputs it to the speakers

   Play()
    Starts or Resumes the playback of a sample

   Stop()
    Stop the playback of a sample

   Send(string ipAdd, int p)
    Creates a new IPEndPoint (from ipAdd and port p) to send audio data
    to through the created socket. Will set the sending flag to true

   ~VoiceClass()
    Class destructor

*************************************************************************/


using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Net;
using System.Net.Sockets;
using Un4seen.Bass.AddOn.Enc;
using Un4seen.Bass;
using Un4seen.Bass.Misc;


namespace _3D_Video
{
    internal class VoiceClass
    {
        #region Member Variables

        //audio input/output devices and audio frequency
        private int recDev, outDev, frequency;

        //record and playback sreams
        int recStream = -1, playStream = -1;
```

```
        //delegate function for the callback on the record sample
        private RECORDPROC recProc;

        //local buffer
        byte[] localbuffer = null;

        //udp socket to send audio buffers
        Socket sock;

        //IPEndpoint to send audio buffer to
        IPEndPoint IEP;

        //bool to determine if voice will be sent
        bool sending;

        #endregion Member Variables

        /// <summary> Initializes a new instance of the VoiceClass</summary>
        public VoiceClass(int rDev, int oDev, int freq)
        {
            recDev = rDev;
            outDev = oDev;
            frequency = freq;
            sending = false;
        }

        /// <summary> Initialize audio devices</summary>
        public void Init()
        {
            //initiate recording device (microphone)
            Bass.BASS_RecordInit(recDev);

            //initiate sound output device (speakers, headphones, etc)
            Bass.BASS_Init(outDev, frequency, BASSInit.BASS_DEVICE_8BITS, IntPtr.Zero,
null);

            this.Stop();
            this.Record();
            this.Play();
        }

        /// <summary> Start recording audio stream from audio input device</summary>
        public void Record()
        {
            recProc = new RECORDPROC(recorder);

            //create handle when starting the recording channel
            recStream = Bass.BASS_RecordStart(frequency, 1, BASSFlag.BASS_RECORD_PAUSE,
                8, recProc, IntPtr.Zero);

            //create a stream and push to a byte stream buffer for playback
            playStream = Bass.BASS_StreamCreatePush(frequency, 1, BASSFlag.BASS_DEFAULT,
                IntPtr.Zero);
        }

        //copy data from recording stream in memory to local buffer
        private bool recorder(int handle, IntPtr buffer, int length, IntPtr user)
        {
            localbuffer = new byte[length];

            //copy data from the buffer pointer to the local buffer
            System.Runtime.InteropServices.Marshal.Copy(buffer, localbuffer, 0, length);

            if (sending)
            {
```

```
                sock.SendTo(localbuffer, IEP);
            }

            return true;
        }

        /// <summary> Output audio stream from buffer</summary>
        public void PlayBuffer(byte[] buffer)
        {
            //playback audio stream
            Bass.BASS_StreamPutData(playStream, buffer, buffer.Length);
        }

        /// <summary> Start/Resume playback of a sample</summary>
        public void Play()
        {
            Bass.BASS_ChannelPlay(recStream, false);
            Bass.BASS_ChannelPlay(playStream, false);
        }

        /// <summary> Stop playback of a sample</summary>
        public void Stop()
        {
            sending = false;
            Bass.BASS_ChannelStop(recStream);
            Bass.BASS_ChannelStop(playStream);
            Bass.BASS_StreamFree(recStream);
            Bass.BASS_StreamFree(playStream);
        }

        /// <summary> Send audio buffer to IP through port</summary>
        public void Send(string ipAdd, int p)
        {
            this.Stop();
            this.Record();
            this.Play();
            sending = true;
            IEP = new IPEndPoint(IPAddress.Parse(ipAdd), p);
            sock = new Socket(AddressFamily.InterNetwork, SocketType.Dgram, ProtocolType.↙
    Udp);
        }


        /// <summary> VoiceClass Destructor</summary>
        ~VoiceClass()
        {
            Stop();
            if (recStream != -1 && playStream != -1)
            {
                Bass.BASS_StreamFree(recStream);
                Bass.BASS_StreamFree(playStream);
            }
            Bass.BASS_RecordFree();
            Bass.BASS_Free();
        }


    }
}
```

```
using System.Reflection;
using System.Runtime.CompilerServices;
using System.Runtime.InteropServices;

// General Information about an assembly is controlled through the following
// set of attributes. Change these attribute values to modify the information
// associated with an assembly.
[assembly: AssemblyTitle("3D Video")]
[assembly: AssemblyDescription("")]
[assembly: AssemblyConfiguration("")]
[assembly: AssemblyCompany("Hewlett-Packard Company")]
[assembly: AssemblyProduct("3D Video")]
[assembly: AssemblyCopyright("Copyright © Hewlett-Packard Company 2008")]
[assembly: AssemblyTrademark("")]
[assembly: AssemblyCulture("")]

// Setting ComVisible to false makes the types in this assembly not visible
// to COM components.  If you need to access a type in this assembly from
// COM, set the ComVisible attribute to true on that type.
[assembly: ComVisible(false)]

// The following GUID is for the ID of the typelib if this project is exposed to COM
[assembly: Guid("88522891-e98b-4d84-9a7b-f38f68eb17b6")]

// Version information for an assembly consists of the following four values:
//
//      Major Version
//      Minor Version
//      Build Number
//      Revision
//
// You can specify all the values or you can default the Build and Revision Numbers
// by using the '*' as shown below:
// [assembly: AssemblyVersion("1.0.*")]
[assembly: AssemblyVersion("1.0.0.0")]
[assembly: AssemblyFileVersion("1.0.0.0")]
```

```
//------------------------------------------------------------------------------
// <auto-generated>
//     This code was generated by a tool.
//     Runtime Version:2.0.50727.1433
//
//     Changes to this file may cause incorrect behavior and will be lost if
//     the code is regenerated.
// </auto-generated>
//------------------------------------------------------------------------------

namespace _3D_Video.Properties {
    using System;


    /// <summary>
    ///   A strongly-typed resource class, for looking up localized strings, etc.
    /// </summary>
    // This class was auto-generated by the StronglyTypedResourceBuilder
    // class via a tool like ResGen or Visual Studio.
    // To add or remove a member, edit your .ResX file then rerun ResGen
    // with the /str option, or rebuild your VS project.
    [global::System.CodeDom.Compiler.GeneratedCodeAttribute("System.Resources.Tools.
    StronglyTypedResourceBuilder", "2.0.0.0")]
    [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
    [global::System.Runtime.CompilerServices.CompilerGeneratedAttribute()]
    internal class Resources {

        private static global::System.Resources.ResourceManager resourceMan;

        private static global::System.Globalization.CultureInfo resourceCulture;

        [global::System.Diagnostics.CodeAnalysis.SuppressMessageAttribute("Microsoft.
    Performance", "CA1811:AvoidUncalledPrivateCode")]
        internal Resources() {
        }

        /// <summary>
        ///   Returns the cached ResourceManager instance used by this class.
        /// </summary>
        [global::System.ComponentModel.EditorBrowsableAttribute(global::System.
    ComponentModel.EditorBrowsableState.Advanced)]
        internal static global::System.Resources.ResourceManager ResourceManager {
            get {
                if (object.ReferenceEquals(resourceMan, null)) {
                    global::System.Resources.ResourceManager temp = new global::System.
    Resources.ResourceManager("_3D_Video.Properties.Resources", typeof(Resources).
    Assembly);
                    resourceMan = temp;
                }
                return resourceMan;
            }
        }

        /// <summary>
        ///   Overrides the current thread's CurrentUICulture property for all
        ///   resource lookups using this strongly typed resource class.
        /// </summary>
        [global::System.ComponentModel.EditorBrowsableAttribute(global::System.
    ComponentModel.EditorBrowsableState.Advanced)]
        internal static global::System.Globalization.CultureInfo Culture {
            get {
                return resourceCulture;
            }
            set {
                resourceCulture = value;
            }
```

```
        }

        internal static System.Drawing.Bitmap _3dvideo_1 {
            get {
                object obj = ResourceManager.GetObject("3dvideo-1", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap background_0 {
            get {
                object obj = ResourceManager.GetObject("background-0", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap background_1 {
            get {
                object obj = ResourceManager.GetObject("background-1", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap background_2 {
            get {
                object obj = ResourceManager.GetObject("background-2", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap background_3 {
            get {
                object obj = ResourceManager.GetObject("background-3", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap background_31 {
            get {
                object obj = ResourceManager.GetObject("background-31", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap background_32 {
            get {
                object obj = ResourceManager.GetObject("background-32", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap ip_1 {
            get {
                object obj = ResourceManager.GetObject("ip-1", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap preview {
            get {
                object obj = ResourceManager.GetObject("preview", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap sendbutton1 {
```

```
            get {
                object obj = ResourceManager.GetObject("sendbutton1", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap sendbutton2 {
            get {
                object obj = ResourceManager.GetObject("sendbutton2", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap stopbutton {
            get {
                object obj = ResourceManager.GetObject("stopbutton", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap voice {
            get {
                object obj = ResourceManager.GetObject("voice", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap voice_0 {
            get {
                object obj = ResourceManager.GetObject("voice-0", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap voice_1 {
            get {
                object obj = ResourceManager.GetObject("voice-1", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap voice_2 {
            get {
                object obj = ResourceManager.GetObject("voice-2", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap voice_f {
            get {
                object obj = ResourceManager.GetObject("voice-f", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }

        internal static System.Drawing.Bitmap webcambutton2 {
            get {
                object obj = ResourceManager.GetObject("webcambutton2", resourceCulture);
                return ((System.Drawing.Bitmap)(obj));
            }
        }
    }
}
```

```
//------------------------------------------------------------------------------
// <auto-generated>
//     This code was generated by a tool.
//     Runtime Version:2.0.50727.1433
//
//     Changes to this file may cause incorrect behavior and will be lost if
//     the code is regenerated.
// </auto-generated>
//------------------------------------------------------------------------------

namespace _3D_Video.Properties {


    [global::System.Runtime.CompilerServices.CompilerGeneratedAttribute()]
    [global::System.CodeDom.Compiler.GeneratedCodeAttribute("Microsoft.VisualStudio.
    Editors.SettingsDesigner.SettingsSingleFileGenerator", "9.0.0.0")]
    internal sealed partial class Settings : global::System.Configuration.
    ApplicationSettingsBase {

        private static Settings defaultInstance = ((Settings)(global::System.
    Configuration.ApplicationSettingsBase.Synchronized(new Settings())));

        public static Settings Default {
            get {
                return defaultInstance;
            }
        }

        [global::System.Configuration.UserScopedSettingAttribute()]
        [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
        [global::System.Configuration.DefaultSettingValueAttribute("8080")]
        public int vPort {
            get {
                return ((int)(this["vPort"]));
            }
            set {
                this["vPort"] = value;
            }
        }

        [global::System.Configuration.UserScopedSettingAttribute()]
        [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
        [global::System.Configuration.DefaultSettingValueAttribute("8070")]
        public int tPort {
            get {
                return ((int)(this["tPort"]));
            }
            set {
                this["tPort"] = value;
            }
        }

        [global::System.Configuration.UserScopedSettingAttribute()]
        [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
        [global::System.Configuration.DefaultSettingValueAttribute("0")]
        public int LeftCam {
            get {
                return ((int)(this["LeftCam"]));
            }
            set {
                this["LeftCam"] = value;
            }
        }

        [global::System.Configuration.UserScopedSettingAttribute()]
        [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
```

```
        [global::System.Configuration.DefaultSettingValueAttribute("1")]
        public int RightCam {
            get {
                return ((int)(this["RightCam"]));
            }
            set {
                this["RightCam"] = value;
            }
        }

        [global::System.Configuration.UserScopedSettingAttribute()]
        [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
        [global::System.Configuration.DefaultSettingValueAttribute("50")]
        public long ImageQuality {
            get {
                return ((long)(this["ImageQuality"]));
            }
            set {
                this["ImageQuality"] = value;
            }
        }

        [global::System.Configuration.UserScopedSettingAttribute()]
        [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
        [global::System.Configuration.DefaultSettingValueAttribute("8060")]
        public int aPort {
            get {
                return ((int)(this["aPort"]));
            }
            set {
                this["aPort"] = value;
            }
        }

        [global::System.Configuration.UserScopedSettingAttribute()]
        [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
        [global::System.Configuration.DefaultSettingValueAttribute("1")]
        public int audioOut {
            get {
                return ((int)(this["audioOut"]));
            }
            set {
                this["audioOut"] = value;
            }
        }

        [global::System.Configuration.UserScopedSettingAttribute()]
        [global::System.Diagnostics.DebuggerNonUserCodeAttribute()]
        [global::System.Configuration.DefaultSettingValueAttribute("0")]
        public int audioIn {
            get {
                return ((int)(this["audioIn"]));
            }
            set {
                this["audioIn"] = value;
            }
        }
    }
}
```

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Windows.Forms;

namespace _3D_Video
{
    static class Program
    {
        /// <summary>
        /// The main entry point for the application.
        /// </summary>
        [STAThread]
        static void Main()
        {
            Application.EnableVisualStyles();
            Application.SetCompatibleTextRenderingDefault(false);
            Application.Run(new Form1());
        }
    }
}
```

```
namespace _3D_Video.Properties {


    // This class allows you to handle specific events on the settings class:
    //  The SettingChanging event is raised before a setting's value is changed.
    //  The PropertyChanged event is raised after a setting's value is changed.
    //  The SettingsLoaded event is raised after the setting values are loaded.
    //  The SettingsSaving event is raised before the setting values are saved.
    internal sealed partial class Settings {

        public Settings() {
            // // To add event handlers for saving and changing settings, uncomment the lines below:
            //
            // this.SettingChanging += this.SettingChangingEventHandler;
            //
            // this.SettingsSaving += this.SettingsSavingEventHandler;
            //
        }

        private void SettingChangingEventHandler(object sender, System.Configuration.SettingChangingEventArgs e) {
            // Add code to handle the SettingChangingEvent event here.
        }

        private void SettingsSavingEventHandler(object sender, System.ComponentModel.CancelEventArgs e) {
            // Add code to handle the SettingsSaving event here.
        }
    }
}
```